

需求和架构 (2)

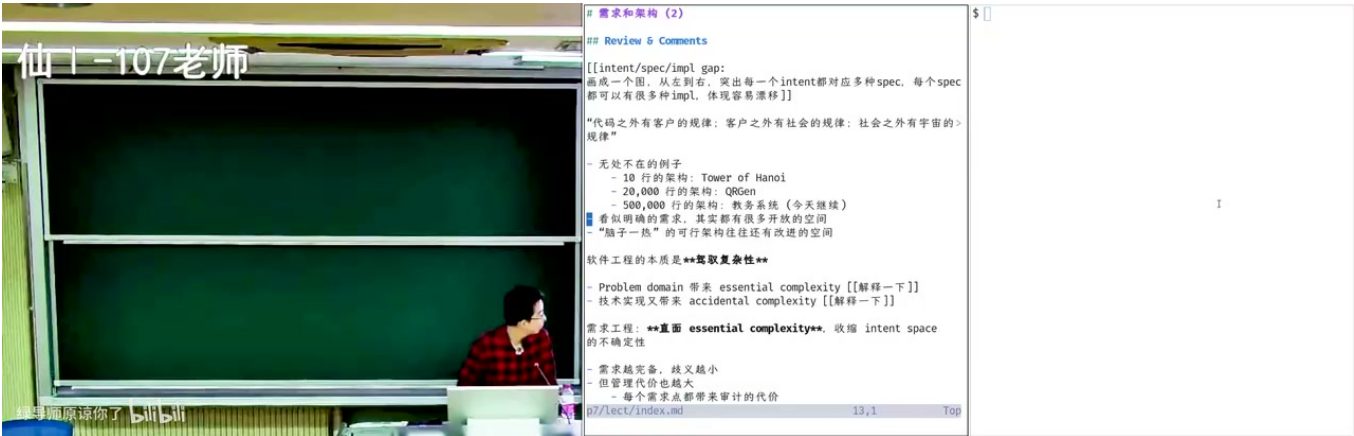
生成式软件工程 · 南京大学
夸克原文图文转录

本稿保留夸克导出的原文、语序和段落，不做摘要或术语改写，尚未逐句听校。时间标记来自 TXT 的段落起点，不代表逐句字幕或精确结束时间。截图按时间归入最近的前置段落，图注保留实际画面时间。

全片 01:39:41 | 138 个段落 | 103 张截图
来源：绿导师原谅你了 · BV1JGAp6LE74

00:00:00

好，今天我们继续讲需求和架构的第二个部分。需求和架构其实有的时候是分不开的。因为我们从第一节杰课就开始说，当我们要做一个软件或者做一个产品，它足够复杂的时候，你就会面对这个intention specification和implementation的。然后我们从最早的那个瀑布模型开始，你也看到我们是怎么样去理解个那gap上用数据依赖的方式，去做一个minimal variable product，然后来控制这样的复杂性。



00:00:34

实际上上一节课我们也讲到，当我们谈论这个需求和架构的时候，有一个原则就是代码之外有客户的规律，客户之外有社会的规律，社会之外带有宇宙的规律。所以你站在哪一个层面去看这个需求，对吧？你可能好上一节课的时候我带大家，大家可能还记得我去用浏览器操作的方式逆向了整个教务系统的这个需求。那不能是整个教务系统就是和教师相关的需求，然后得到了一份这个需求规格的说明书，然后那个说明书上写的很好，对吧？这个是符合软件工程实践，也符合做产品的这样的一个做需求的方式。但是你作为整个系统的设计者或者整个系统的架构师，你可能要站在这个需求之外来去想。这些你当然是可以说好，我现在就选一个价格，选一个数据库，然后就把这些需求都实现了。你可以交付一个百分之百满足客户需求的产品。



画面 00:01:00

00:01:37

但是这个产品很有可能会在未来的维护当中陷入一个长期的泥潭。也就是说你在做任何设计的时候，做的那一刻起，你就背上了一定程度的技术债，然后你会一直一直要背下去。而且大家如果留意编程，就是在编程的时候注意过很多细节的话。其实我们从十行的代码，最小的一份代码。今天其实我还有几个例子。最小的代码到你可能要做一个50万行的代码、100万行代码、1000万行的代码。



画面 00:02:00

00:02:05

你要做一个大系统的时候，它其实这个架构都是无处不在的。而且上节课大家还记得有一个图说，我们从最初的时候，什么也没有的时候，我们就需要做决定。然后一旦我们做决定，那它就会影响未来的走向。所以其实你的软件架构的设计通常来讲都是有很大的改进空间的。所以你们如果做实验，我应该这两天会把下一个实验放上来。

00:02:30

做第一个实验的时候，你可以直接像我那次上课一样，直接把需求复制粘贴进去，让AI帮你直接做一个东西，然后你不停的跟他聊天修改。但是大家要记得，你如果马上第一反应说这个东西可以这样做吗？可以这样设计数据库，你的第一反应应该是，也许我可以回过头来再多想一想，可能会有更好的方案。好，这是上节课内容的一点回顾。我们谈软件工程的时候，bro就说了，我们软件工程的本质就是驾驭复杂性。

架构无处不在

“ 代码之外有客户的规律；客户之外有社会的规律；社会之外有宇宙的规律 ”

从 10 行到 500,000 行

- 10 行的架构：Tower of Hanoi
- 20,000 行的架构：QRGen
- 500,000 行的架构：教务系统（今天继续）

可行架构仍有改进的空间

- “脑子一热”的可行架构往往还有改进的空间

2/5

架构无处不在

“ 代码之外有客户的规律；客户之外有社会的规律；社会之外有宇宙的规律 ”

从 10 行到 500,000 行

- 10 行的架构：Tower of Hanoi
- 20,000 行的架构：QRGen
- 500,000 行的架构：教务系统（今天继续）

可行架构仍有改进的空间

- “脑子一热”的可行架构往往还有改进的空间

2/5



软件工程的本质：驾驭复杂性

Problem Domain → Essential Complexity

- 所选问题本身要求处理的复杂性：课程容量、先修要求、时间冲突。
 - 即使换成人工办理，这些约束仍然存在。

技术实现 → Accidental Complexity

- 实现方式额外引入的复杂性：重复数据、格式转换、缓存同步、部署配置。
 - 业务规则没变，换一种实现，就可能省掉许多麻烦。
- 这一区分来自 Brooks 的 No Silver Bullet：区分问题本身的困难与表达、实现它时附带的困难。

8/5

00:03:07

将来我们在谈需求的时候，你看到那个需求规格说明书的时候，你就知道现实中的软件是非常复杂的对吧？非常复杂，那有一些复杂性是没有办法消除的。这个problem domain里面的essential complexity？它的问题就是这么复杂。我们要考虑这个课程的容量，先修的要求，时间的冲突，他要排一个对吧？还要排课，还要考虑教室的这个情况。我们的系统里面还支持，比如说我们的老师和同学都可以借教室。这个物理世界就是复杂的，而这个物理世界在信息世界里面的投影，它的这个复杂性是没有办法消除的那没有办法消除的时候，我们其实是用架构软件的架构来更好的去管理这样的复杂性。

00:03:57

除此之外，我们刚才讲到技术债，对吧？你在做任何实现决定的时候，如果我们要用这个数据库系统或者你就用文件系统管理的时候，你都会引入一个trade off space，对吧？你有时候你文件系统你你是管理起来是很方便，我想hack它也很容易。但是你带来代价，它的性能可能远远不如数据库，而且它有并发控制的问题，文件系统本身是不支持这种原子性的事物，就是all nothing对吧？我要我可以读多个数据，写多个数据，并且整个读和写看起来是atomic的，就像我上了一把锁，然后stop the world一样。文件系统是没有这个保证的。



软件工程的本质：驾驭复杂性

Problem Domain → Essential Complexity

- 所选问题本身要求处理的复杂性：课程容量、先修要求、时间冲突。
 - 即使换成人工办理，这些约束仍然存在。

技术实现 → Accidental Complexity

- 实现方式额外引入的复杂性：重复数据、格式转换、缓存同步、部署配置。
 - 业务规则没变，换一种实现，就可能省掉许多麻烦。
- 这一区分来自 Brooks 的 No Silver Bullet：区分问题本身的困难与表达、实现它时附带的困难。

8/5

00:04:37

虽然其实有比较疯狂的人想要在操作系统里面实现这种文件系统级的transaction。数据库能够帮你实现transaction，但是它带来的缺点就是你每次调试的时候都需要经过一个中间层，对吧？你需要看数据库的时候，你不能像文件系统一样使用所有的unix工具去组合它。理论上说也可以对吧？你还是可以把和其他的工具组合起来。那样就会比如说让A阵做，就会引入一个额外的转换层，然后这些都是你做任何技术的选型都会有的这种accidental complity，这两个要分开。然后其实好的，就如果你们在长期的学习各种各样好的软件架构的过程中，你们就会看到什么样的什么时候应该用什么样的架构来减少这个complity，以及怎么样一个好的系统是怎么应对这个世界上面的这种essential complity的。



00:05:35

对所以其实一定程度上需求工程就是去直接面对这个系统里面的essential complexity。他承认软件系统就是复杂的，然后我需要不停的和客户去沟通，把需求我到底要什么这件事情想清楚。不管是你们看到现在的这个就经典的对吧？我们说让gp t写了一个需求规格说明书，还是你今天可能会有那种map co的，那种grime的skill，我其实推荐大家去试用一下。你你你做任何一个小的东西的时候，对吧？



00:06:09

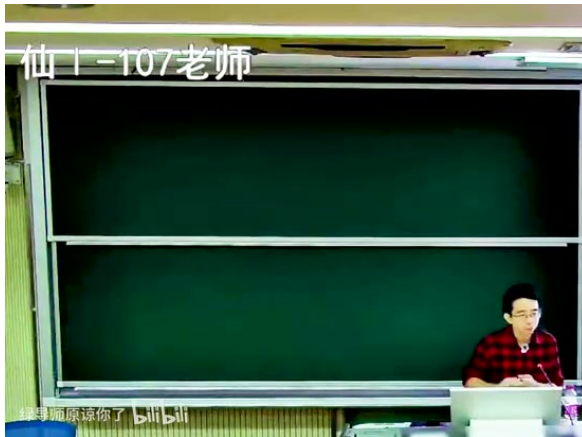
比如说我就要做一个购物车，打个比方，然后那就会把你所有的细节都给你问一遍，你到底要怎么做authentication。这个authentication你是接入一个第三方的那种微信公众号的认证，还是你要自己做一个login page？然后这时候其实你的每一个选择在这个需求其实这个还是需求层面的东西，你的每一个选择都会。都会有很大的影响。

00:06:36

如果你直接比如说就把这个需求直接交给gp做，说我要login，那gp就会给你猜一个对吧？猜一个的时候他可能会说你需注册用户。好，注册用户马上带来的问题是用户要输入用户名和密码，这个肯定没问题。第一次的时候密码会需要一个confirmation。

00:06:54

好，接下来就是你要不要做一个忘记密码的功能。如果你没有忘记密码的功能，用户一定会忘掉密码的那对于用忘掉用密码的用户来说，这就是一个非常糟糕的体验。因为它的数据留在里面了，它没有办法再access了。然后如果你要解决这个忘记密码的功能，你要么要给他发短信，要么给他发邮件。好，发邮件的时候你用什么来发邮件对吧？



00:07:18

然后你当你把这些问题不断的往后想的时候，你你你当时觉得ok因为所有人都有一个用户密码登录，所以我的软件也理所应当做一个用户密码登录。但你可能做这个设计或者说需求，你做这件事情的时候，你就已经进入了一个泥潭，对吧？泥潭。所以有时候你想哈我还是用微信的登录比较好，对吧？微信或者别的，因为你看到现在的，比如说你们注册一个网站，很多都支持微信或者支付宝登录，因为这是一个更方便的模式。

00:07:50

好，那怎么办呢？你要交钱，而且个人对这个对个人开发者可能不是很友好。你可能需要一个公司实体他才能注册，因为你你个人承担责任的能力和公司是是不一样的。公司某种程度上你需要遵守一定的东西才能注册，对吧？然后这就就变得非常复杂，就仅仅是登录这一件事，你就有无穷的细节，然后进入进去以后又有各种各样的其他的细节。所以在需求里面，一方面我们可以深度优先，其实这两个是不矛盾的。第一个是我们可以深度优先去做一个MVP，然后这MVP可以快速的让你知道那些最重要的需求功能点，你到底是不是满足了，满足到什么程度。然后这个软件比如说需求要不要重做？



画面 00:08:00

00:08:37

第一第二轮的重做，以及你的思维方式同样也需要一定的广度优先。我们有很多的决策，我们的登录框到底是选哪一种方式来实现，然后我们的界面到底是怎么样布局，第一个首页到底怎么怎么展现，这些都是需求space里面的里面的东西。然后直到总有一天，你还是需要把这个需求以某种方式写出来的。理论上说就是代码本身就是需求。因为我们可以让大语言模型，就你打比方，你可以把所有的文档都删掉，一个实际的项目，然后你就让AI去运行代码，读代码，然后他可以把些需求点给还原出来，但还原出来的这个需求点就会非常长对吧？它没有一个可能也可以有一个从顶到上的理由的设计，对吧？

需求工程：直面 Essential Complexity

收缩 Intent Space 的不确定性

- 需求越完备，歧义越小
 - “课程容量 100 人”：候补算不算？跨院预留名额能否借用？

但管理代价也越大

- 每个需求点都带来审计的代价
- 换到今天，是 token 开销（和模型潜在的错误发挥）
 - 每补上一条约束，也多了一条需要持续核对的承诺。



画面 00:09:00

00:09:30

我们的整个软件系统的设计需求的宗旨是不要搞太复杂，要简化复杂性，要管理好我们的个就如果我我我实现起来麻烦的东西，我最好是交给一个第三方来管理，那这就是你的你的设计哲学。然后也可能你的产品它不是这样，它就是需要用户名和密码，然后你就想要管理这个用户的数据。比如说你可能就是想要偷这个用户的密码，去去构建自己的密码库。当然这个事情肯定是不合规的，但在你体量足够小的时候，你可能你就想想要做做这件事儿。所以这其实是一个吹悼，你可能需要在深度优先和广度优先之间达到一个比较好的平衡。



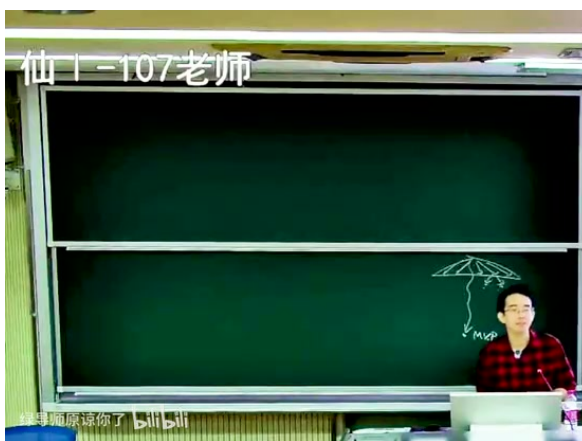
画面 00:10:00

00:10:21

那就只能做动手做对吧？你一方面你去想一个软件系统，它它为什么做的这个设计？然后如果我换另外一个设计，它会变好还是变坏，对吧？我不知道你们上节课看完了教务系统以后，有没有想想如果你来实现这个教务系统，你你会你会怎么做对吧？怎么做？所以这是一些这些需求工程，这个好像没有特别的办法，你只能看得多做的多。

00:10:45

所以我今天当然今天也会讲一些具体的系统的实现。然后需求之外就是架构，架构有两个刚才讲到有两个主要的作用，第一个是分解以三手。就有些东西是确实是没有办法避免的，但是有可能把它隔离在不同的模块里，然后把这个模块之间做一个很好的接口的切分。比如说数据库就是一个非常好的架构设计对吧？M1C我今天会讲M1C，当你把数据放到数据库以后，你就再也不用关心数据是怎么存储的了。它给你一个很好的contract，尤其是你使用比如说云云厂商的数据库的时候，他一般都会有给你备份的承诺，就是你数据不会丢的，最多就是可能一段时间出问题了，然后他可能会降低你的availability，但是你的备份很快就能顶上，然后这个数据库访问起来几乎是没有任何的感觉的。这样你对数据的可靠性，然后数据访问的原子性，那这些你都不用担心了，你就可以更好的关注在你的业务需求上。



画面 00:11:00

00:11:53

当然数据库本身是一个通用的system，你可以把一些你的逻辑就下放给数据库。然后同样你在你自己的软件系统里面也可以设计不同的模块。比如说你的信息系统对吧？学校最最希望的是打造一个全部连打通的信息系统。你填任何信息，你不需要在这里填一次那里填一次，这里填一次，那里填一次。

需求工程：直面 Essential Complexity

收缩 Intent Space 的不确定性

- 需求越完备，歧义越小
 - “课程容量 100 人”：候补算不算？跨院预留名额能否借用？

但管理代价也越大

- 每个需求点都带来审计的代价
- 换到今天，是 token 开销（和模型潜在的错误发挥）
 - 每补上一条约束，也多了一条需要持续核对的承诺。

架构设计

分解 Essential Complexity; 减少 Accidental Complexity

- 好的架构能更容易地实现需求、承受需求的变更
 - 把容量、资格、时间冲突分开建模，让每一部分都能单独理解。
 - 让页面、批处理共用规则接口，减少重复实现与口径不一。

收缩 Spec / Impl Space 的可能性

- 明确模块职责、数据归属和接口约定，排除大量随意组合。
- 一个实用判断：需求改一处，需要跟着改多少处？



架构设计

分解 Essential Complexity; 减少 Accidental Complexity

- 好的架构能更容易地实现需求、承受需求的变更
 - 把容量、资格、时间冲突分开建模，让每一部分都能单独理解。
 - 让页面、批处理共用规则接口，减少重复实现与口径不一。

收缩 Spec / Impl Space 的可能性

- 明确模块职责、数据归属和接口约定，排除大量随意组合。
- 一个实用判断：需求改一处，需要跟着改多少处？

00:12:19

然后我们花了很长的时间才完成了这个统一身份认证和打通。但学校里面其实还有很多的系统可能你们没有用过。比如说你们不涉及财务系统，对吧？你们你们你们不从学校领工资，然后也不管理这个，也不管理项目。那你们可能会给老师，如果研究生的话，可能会给老师报销过这个。一些单据，那就是非常麻烦的一个财务系统。

00:12:40

财务系统就完全没有和现在的系统打通，这也是可以理解的。因为财务系统里面有很多领域知识，对吧？他有大量的领域知识，他要把账做平，接受审计。这些功能本身是在他的领域里。

00:12:55

现在财务系统和其他的部分还没有很好的连起来。比如说实现数据的共享通知的推送这些。但是其实这是一个比较好比较我们希望得到的一个功能，也希望是需求能够需求和架构能够做到做到这一点。然后架构能够在一定程度上就是收缩这个specification和information space的可能性。我们的需求对吧？需求就是去收缩intenspace的不确定性，我们会会漂移对吧？在这里我们的intent到spec到implementation，那每一个intent的可能性都会有多种不同的spec的实现，然后每一个specification又可以有多种implementation的实现。所以为了减少这样的漂移需求，实际上是收缩了inspace的不确定性，然后架构是收缩了这个spec和intation space的可能性，但你一旦确定的时候，你就会背上一定程度的技术债。



架构设计

分解 Essential Complexity; 减少 Accidental Complexity

- 好的架构能更容易地实现需求、承受需求的变更
 - 把容量、资格、时间冲突分开建模，让每一部分都能单独理解。
 - 让页面、批处理共用规则接口，减少重复实现与口径不一。

收缩 Spec / Impl Space 的可能性

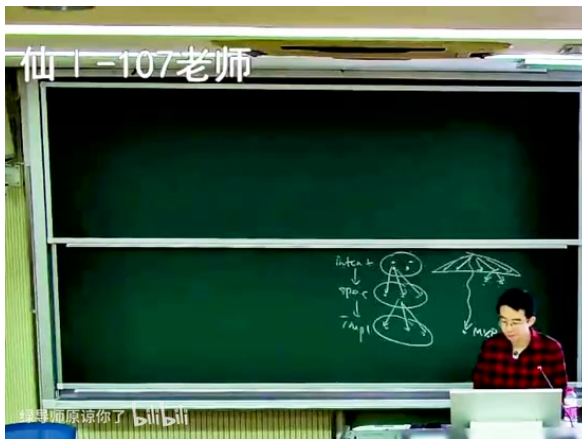
- 明确模块职责、数据归属和接口约定，排除大量随意组合。
- 一个实用判断：需求改一处，需要跟着改多少处？



画面 00:14:00

00:14:14

这个是上一节课的内容，然后好，但还是觉得不知道我说了什么，对吧？不知道说了什么。然后我就发现一件很有意思的事儿，我最近读了又重新读了一些老paper。然后我发现在你这这其实在软甚至软件工程诞生的初期，甚至更早的时候，我们其实已经有很多的大家们思考过。比如说软件或者程序应该怎么构造。哪怕如果你去看它的原文的话，我这里有它的maybe I one。这是make notes on data structure，这是叫结构化程序设计。我还是用PDF来打开它，对吧。



画面 00:15:00

00:15:23

很很有意思的一本老书。你看到里面的里面的程序，你就知道这不是一个现在意义上的软件工程的事。你看到它都是在讲这种相对比较小的程序，你的这个你还能看到这样的流程图，它讲顺序结构、选择结构、循环结构。它的argument是其实我们不需要go to，在那个时候他们还在争论对吧？Go to is harmful.

00:15:52

你只需要用这个顺序选择循环这种diagram的方式，就可以表示任何的程序逻辑。他们还在argue这种它是一个比较小的函数的这个事情。然后我们可以找到里面的一些示例的程序，让我们来找一下。这看起来还是非常简单的程序，但是这个程序开始出现一些结构了对吧？它反正这个书写的很好，那我们当然不应该用这种传统的方式来读它，对吧？

需求工程：直面 Essential Complexity

收缩 Intent Space 的不确定性

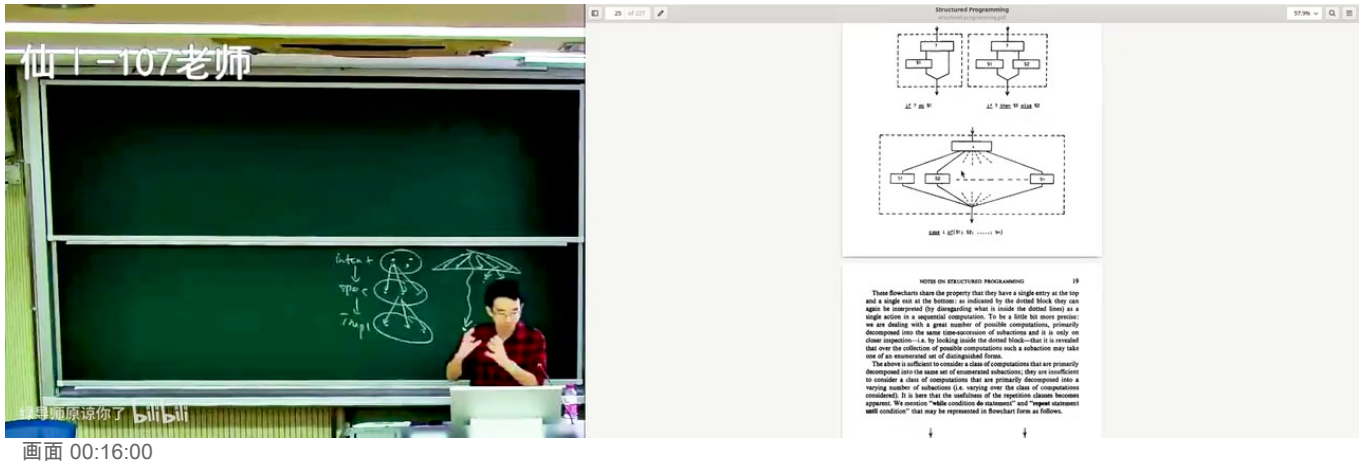
- 需求越完备，歧义越小
 - “课程容量 100 人”：候补算不算？跨院预留名额能否借用？

但管理代价也越大

- 每个需求点都带来审计的代价
- 换到今天，是 token 开销（和模型潜在的错误发挥）
 - 每补上一条约束，也多了一条需要持续核对的承诺。

8/5

```
$ eza --git --group-directories-first --icons=auto
assets  p1  p4  p7  AGENTS.md  README.md
docs    p2  p5  scripts  CLAUDE.md  TODO.md
guest   p3  p6  tentative Makefile
$ cd ..
$ cd readspace/
$ eza --git --group-directories-first --icons=auto
article build morning-paper AGENTS.md
arxiv    lectures review    CLAUDE.md
books    magazine scripts
$ ./scripts/read-tui books/structured-programming/notes-on-data-structuring-hoare.md
$
$ ./scripts/read-tui books/structured-programming/gramming/front-matter.md
gramming/hierarchical-program-structures-dahl-hoare.md
gramming/notes-on-data-structuring-hoare.md
gramming/notes-on-structured-programming-dijkstra.md
```

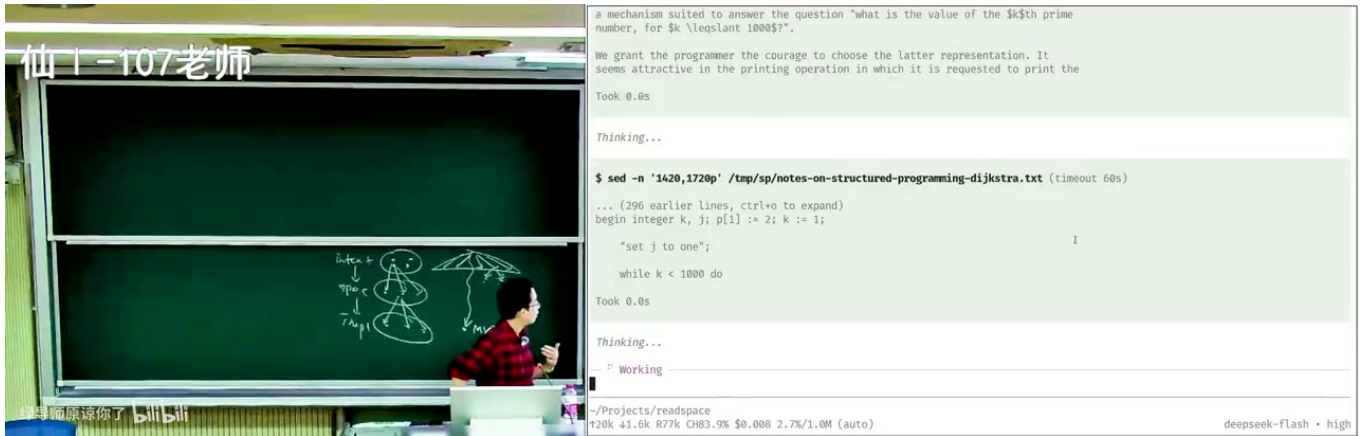


画面 00:16:00

00:16:36

我想阅读这个结构化程序设计，能先帮我总结一下全书。怎么说呢？读书是不可避免的对对大家来说，但是我突然发现在今天读书比以前要变得容易了。前提是你用上正确的好用的AI工具。我因为我我就前两天

的时候，我干了这样的一件事。



画面 00:17:00

00:17:09

我给每一个我的博士生的建议，都我都我都告诉他们，你需要去读一些这种经典的文献。比如说因为我自己做这个操作系统PR软件工程相关的，我就会说这三个领域可能从1980年到2010年是一个golden age。然后在那个后面以后做的东西都更加就越做越小，这个算是比较越做越窄了。那但是在比如说1980年到2011年的时候，他还有很多这种天马行空的想法。在这个系统还不存在的时候，他怎么把这些东西给构建出来。

00:17:47

然后我就昨天，还是前两天，我让我的bot kser送了一些grobobot额度，他有一台云电脑在后面，然后我就说好，你去帮我收集1980年到2010年的我觉得值得读的systems的文献。然后他找1000个他就找1000个，他大概每个领域就找了两三百篇。然后我我因为我基本上那些文献大部分都读过，然后我就仔细的核对那里面的列表，然后发现他找列表还是非常好的，就是基本上都是我觉得很多文章也是我会推荐的文章，然后这就是一件非常恐怖的事情。

仙一-T07老师



画面 00:18:00

读完了三篇的目录与正文脉络 (books/structured-programming/, 1972 年 Academic Press, APIC No. 8)。先说结论：这本书不是一本“教材料”，而是三篇各自独立、互相呼应的宣言式长文，合起来回答同一个问题——人的头脑很小，怎样组织复杂度，才能造出并相信自己所造出的大型程序？

一、全书框架图

结构化程序设计 (1972)

- I. Dijkstra 《结构化程序设计笔记》
“怎么造程序”
程序/算法结构
- II. Hoare 《数据结构化笔记》
“怎么造数据”
数据/类型结构
- III. Dahl & Hoare 《层次化的程序结构》
“把两者合成概念”
类·对象·概念层次

共同主线：用「抽象 + 层次分解」把
“人不能一次理解的东西”变成“人可以逐层理解的东西”

- 心智工具：枚举 / 数学归纳 / 抽象
- 方法：分而治之、逐步细化、自顶向下 vs 自底向上
- 判断：可证明正确 + 可修改 (程序家族)
- 产物：程序 = 一串“珍珠”(pearl) / 数据 = “类型-表示”分离

二、第一篇：Dijkstra —— 程序如何被理解

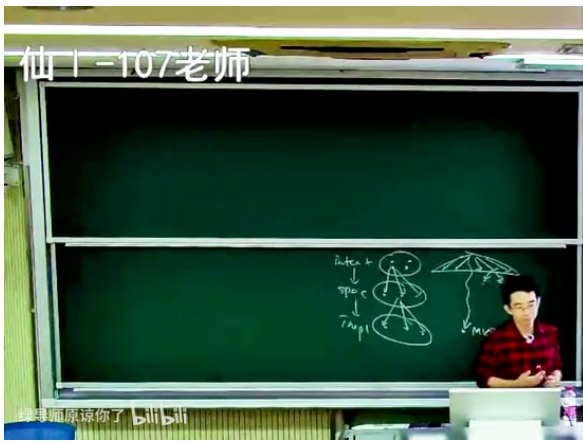
00:18:24

然后我我现在我现在比如说我开车的时候，我就会打开一个GPT或者打开一个cloud，我会跟他聊天，然后聊天的时候我可以说好聊一篇paper。因为那我的paper会像，我的paper会在这里被锁被被锁引上。对我的paper会被索引。然后这样的话我的AI I其实实际上是可以看到这个paper的全文的。

00:18:57

然后我只要把paper丢进去，然后接进去以后，我就可以比如说一边开车一边今天这个paper是怎么读的，我有一次教过大家怎么读paper。我从你自己知道的知识开始，把你的把你的领域不停的往外扩张。你要解决什么问题？一个你可能你不知道的问题，然后用了你什么样不知道的手段，然后得到了什么样的结论。然后和你所有的知识连成一个网络，然后你最终就理解了这个paper，然后它这里面就有很多我觉得真的是能给人很多启发的东西。

仙一-T07老师



画面 00:19:00

六、阅读建议

- 若想通读，先 Dijkstra 第 2、3、7、9、14 章抓主线，再回读 12、13、17 章看虚拟机与递归。
- Hoare 前两章 + 最后一章 (公理化) 最像“方法论”：中间 3-10 章可当数据结构手册查阅。
- Dahl & Hoare 若不吃 SIMULA 语法，读 1、6、7 章即可理解“前缀-继承”和“概念层次”。
- 三篇都在反抗同一个敌人：**程序是一长串符号、改程序就是改文本**的世界观。

需要的话，我可以按 reader skill 的方式，帮你逐篇精读其中某一部分 (比如 Dijkstra 的“pearl/necklace”或八皇后)，并把关键原图提取出来一起看。

Recap: 把这本书当作“三篇问题散文”而非教材来看——Dijkstra 讲如何用逐步细化和分层虚拟机组织算法，Hoare 讲如何用类型抽象和推论表示组织数据，Dahl & Hoare 则用 SIMULA 的类、前缀、协程把两者合成概念层次：三条线由“人的智力有限、必须分而治之”这一共同前提串起，最终汇入结构化编程、抽象数据类型与面向对象。

~/Projects/readspace
169k 47.3k R1.0M Q97.9% \$0.035 7.7%/1.0M (auto) deepseek-flanotes-on-structured-programming-dijkstra.md 1-30/4862 0%

00:19:35

首先这个打家上来就说这个。Our inability to do much. 我们人本身刚才我们讲了，比如说需求就是复杂的，然后我们要做架构。但实际上为什么需要做这件事呢？就是因为我们的能力是有限的，我们是一个参数有限的大模型。对我们还是承认我们自己是大模型，所以这里面 I have a very small head, and I had better to learn to live with it and to respect my limitations and give them full credit, rather than trying to ignore.



画面 00:20:00

00:20:07

所以这里就有很多有意思的想法，对吧？他说用抽象加上层次的分解structure programming，你看起来这个概念已经过时了。就是我们现在是object oriented对吧？面向对象，然后还有很多这种比较fancy的框架，你们用前端的，可能用reac或者view。这两批社区的开发者至今还在圣诞对吧？就像以前vim和e max的开发者还在圣战对吧？虽然他们两个都越像了，但你看到这些今天所有的这些东西在过去，其实他们已经想想过一遍了，他需要用一个层次化的分解方式，把人不能依次理解的东西变成人可以逐层理解的东西。

00:20:55

然后这里面有很多我真的是特别喜欢的观点。比如说我看我有没有sli呢，有一个ormental AI呃枚举归纳和抽象。这个Parman最重要的，你就就我们的人脑可以干的就是这三件事情。你如果去看那里面的案例，它今在今天看来都是小程序。刚才带大家看过，但是他做了一个比喻，他说我们要做一个程序，或者你就想象成做一个软件系统，你就要做，你做的其实是一个necklace，它要做一个项链，然后项链上的珍珠，每个项链上的小珍珠把它串起来得到一个软件。这个比其实就是在分解，就要告诉你分解复杂性它怎么样，对吧？把一个人类不能理解的东西变成人类可以逐个理解的东西。好，然后你看这样一本书，你就这样想，老师可能会在课上推荐大家也许是比如说我在课上推荐大家读一本书，也许是我我们的你们看某一本教科书的时候，后面有一个角注说我的这个想法教科书的想法来自于这一本书。



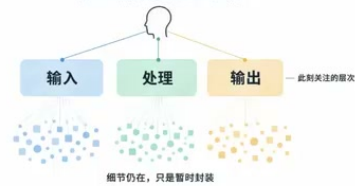
画面 00:21:00

On Our Inability to Do Much

- *Structured Programming* (O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare) ©, 1972

“...I have a very small head and I had better learn to live with it and to respect my limitations and give them full credit, rather than to try to ignore them...”

尊重有限的注意力



118

Our Mental Aids

枚举、归纳、抽象

- Dijkstra 的 Part I 讨论了 our mental aids: 枚举、归纳、抽象。
- 虽然讨论的案例在今天看来都是“小程序”，但 necklace (pearls) 的比喻就是在分解复杂性。

Necklace (Pearls): 设计决策之间的依赖

- 每颗 pearl 承载一项独立设计决策：决定的先后，会改变其他层必须知道多少细节。
- 过早把 image 解释成 lines，中间各层就都要理解 lines：推迟表示决策，更多层才能独立于它。原文：[On What We Have Achieved](#)

119



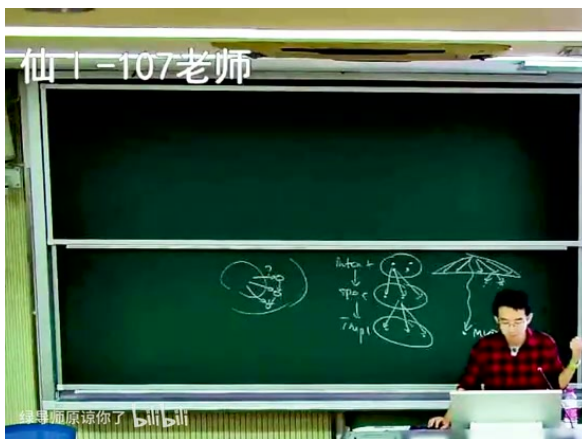
画面 00:22:00

00:22:10

我不知道你们有没有留意过，如果你们读一些比如说经典的这个programming language，或者软件设计的时候，它可能都会引用这个。对，因为这个太有名了，都会引用到这本书。然后在引用这本书的时候，你们有两种选择，一种是这个，当然我觉得我读书的时候都基本上也都必须要面临这个选择。就是看一下大家好像写过这本书，忽略它。因为如果我要把这本书找挖出来，然后再去读它，这件事情对我来说会浪费太多的时间。但对大家来说，你们现在就可以？一直是可以直接去用原始的最原始的第一手信息，去按照你自己想要的方式来处理。

00:22:57

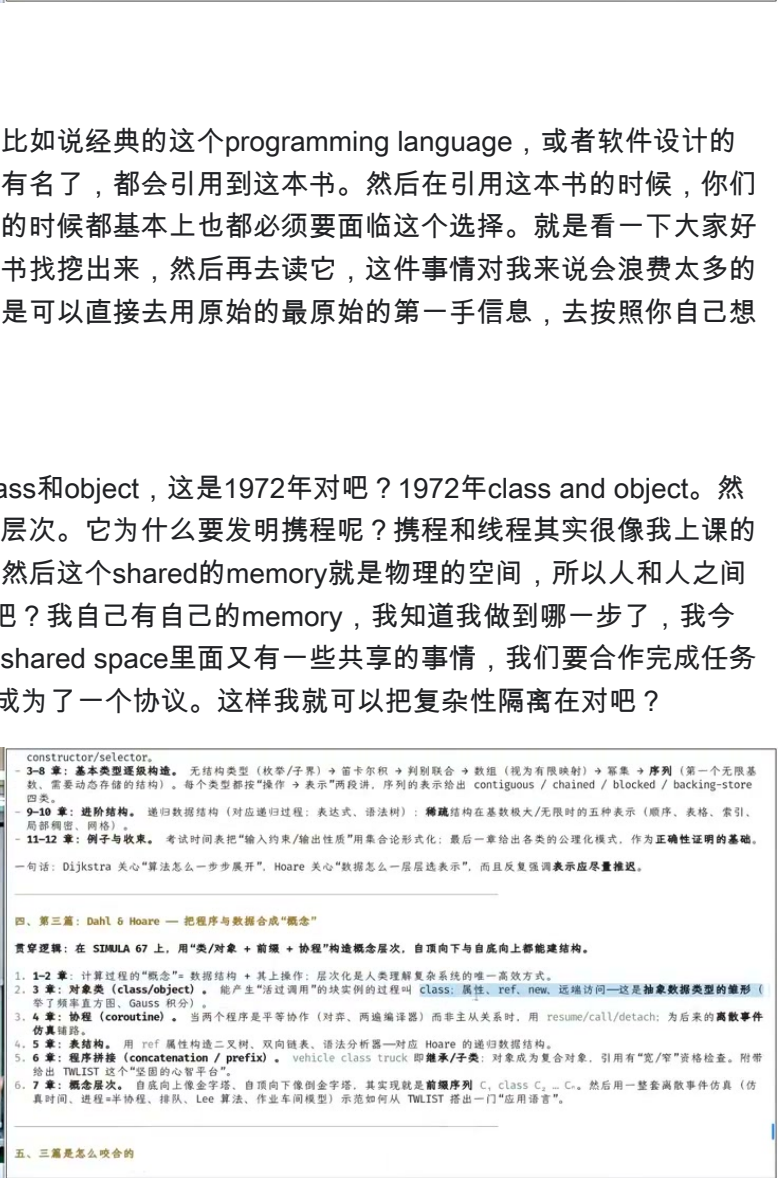
好，这里其实还是有很多东西，你看他讲了class和object，这是1972年对吧？1972年class and object。然后携程他考虑的是怎么样构造各种各样的概念层次。它为什么要发明携程呢？携程和线程其实很像我上课的时候，线程打什么？比方线程，就比方成人，然后这个shared的memory就是物理的空间，所以人和人之间是协作的，每个人都有他自己的memory，对吧？我自己有自己的memory，我知道我做到哪一步了，我今天要干什么，明天要干什么。但同时我在这个shared space里面又有一些共享的事情，我们要合作完成任务，那我们人和人之间的通信communication就成为了一个协议。这样我就可以把复杂性隔离在对吧？



画面 00:23:00

00:23:47

我我可以把一个工作我可以把一个工作分解成三条线，它们最终共同完成工作。然后他们之间可以通过一个数据结构来或者一个什么东西来来共享？他们可以不断的向前进度进展可以不断的向前，然后他们要互相通信来完成整个任务？然后他举了一些这样的博弈下棋的例子，还有编译器的例子。然后有一个概念层次的金字塔。你看最终你如果今天去看那些好的设计的好的软件软件系统的话，我们做的事情无非就是这个书里面其实已经讲过的事情。所以我们让AI来带，我们就今天假设大家前面的东西都已经多多少少知道，但其实你们不知道对吧？



仙！-107老师



画面 00:24:00

constructor/selector.

- 3-8 章：基本类型递归构造。无结构类型（枚举/子界）→ 笛卡尔积 → 判别联合 → 数组（视为有限映射）→ 链表 → 序列（第一个无限基数，需要动态存储的结构）。每个类型都按“操作 → 表示”两段讲。序列的表示给出 contiguous / chained / blocked / backing-store 四类。
- 9-10 章：进阶结构。递归数据结构（对应递归过程：表达式、语法树）；稀疏结构在基数极大/无限时的五种表示（顺序、表格、索引、局部稠密、网络）。
- 11-12 章：例子与收束。考试时间表把“输入约束/输出性质”用集合论形式化；最后一章给出各类的公理化模式，作为正确性证明的基础。

一句话：Dijkstra 关心“算法怎么一步步展开”，Hoare 关心“数据怎么一层层递表示”，而且反复强调表示应尽量推迟。

四、第三篇：Dahl & Hoare —— 把程序与数据合成“概念”

贯穿逻辑：在 SIMULA 67 上，用“类/对象 + 前缀 + 协程”构造概念层次，自顶向下与自底向上都能递归结构。

- 1-2 章：计算过程的“概念”= 数据结构 + 其上操作；层次化是人类理解复杂系统的唯一高效方式。
- 3 章：对象类（class/object）。能产生“活过调用”的块实例的过程叫 class：属性、ref、new。远端访问一这是抽象数据类型的雏形（除了频率直方图、Gauss 积分）。
- 4 章：协程（coroutine）。当两个程序是平等协作（对弈、两遍编译器）而非主从关系时，用 resume/call/detach：为后来的函数事件仿真铺路。
- 5 章：表结构。用 ref 属性构造二叉树、双向链表、语法分析器——对应 Hoare 的递归数据结构。
- 6 章：程序拼接（concatenation / prefix）。vehicle class truck 即继承/子类；对象成为复合对象，引用有“宽/窄”资格检查。附带给出 TWILIST 这个“坚固的心智平台”。
- 7 章：概念层次。自底向上像金字塔，其实现就是前缀序列 C, class C₁ ... C_n。然后用一整套函数事件仿真（仿真时间、进程=半协程、排队、Lee 算法、作业车间模型）示范如何从 TWILIST 搭出一门“应用语言”。

五、三篇是怎么咬合的

00:24:50

你们可能只是在上操作系统课的时候第一次听说了 corotining。然后coting在在今天的意义下，更多的是一个为了性能实现的轻量级的线程。比如说够的那种携程的模型，它就是为了让你能够像现成一样用的很开心。同时它的每一个携程的切换和不keeping的开销都非常小。来因为基闻已经是现成模型了，那个时候你看几乎没有它没有现成的现成的模型。但是如果你回到它设计这些机制的基础上，就回到那个时间点，你会知道他们其实是在应对软件世界里面的复杂性。好，那我们就试一试一试试，这实因为我也没有仔细读过。

仙！-107老师



画面 00:25:00

constructor/selector.

- 3-8 章：基本类型递归构造。无结构类型（枚举/子界）→ 笛卡尔积 → 判别联合 → 数组（视为有限映射）→ 链表 → 序列（第一个无限基数，需要动态存储的结构）。每个类型都按“操作 → 表示”两段讲。序列的表示给出 contiguous / chained / blocked / backing-store 四类。
- 9-10 章：进阶结构。递归数据结构（对应递归过程：表达式、语法树）；稀疏结构在基数极大/无限时的五种表示（顺序、表格、索引、局部稠密、网络）。
- 11-12 章：例子与收束。考试时间表把“输入约束/输出性质”用集合论形式化；最后一章给出各类的公理化模式，作为正确性证明的基础。

一句话：Dijkstra 关心“算法怎么一步步展开”，Hoare 关心“数据怎么一层层递表示”，而且反复强调表示应尽量推迟。

四、第三篇：Dahl & Hoare —— 把程序与数据合成“概念”

贯穿逻辑：在 SIMULA 67 上，用“类/对象 + 前缀 + 协程”构造概念层次，自顶向下与自底向上都能递归结构。

- 1-2 章：计算过程的“概念”= 数据结构 + 其上操作；层次化是人类理解复杂系统的唯一高效方式。
- 3 章：对象类（class/object）。能产生“活过调用”的块实例的过程叫 class：属性、ref、new。远端访问一这是抽象数据类型的雏形（除了频率直方图、Gauss 积分）。
- 4 章：协程（coroutine）。当两个程序是平等协作（对弈、两遍编译器）而非主从关系时，用 resume/call/detach：为后来的函数事件仿真铺路。
- 5 章：表结构。用 ref 属性构造二叉树、双向链表、语法分析器——对应 Hoare 的递归数据结构。
- 6 章：程序拼接（concatenation / prefix）。vehicle class truck 即继承/子类；对象成为复合对象，引用有“宽/窄”资格检查。附带给出 TWILIST 这个“坚固的心智平台”。
- 7 章：概念层次。自底向上像金字塔，其实现就是前缀序列 C, class C₁ ... C_n。然后用一整套函数事件仿真（仿真时间、进程=半协程、排队、Lee 算法、作业车间模型）示范如何从 TWILIST 搭出一门“应用语言”。

五、三篇是怎么咬合的

00:25:43

应该是part three的。面向上课的本科生？看一看他对这件事情的理解，这个时候他会去读原文，然后这个原文哪怕你就让它翻译，然后去掉一些你觉得不太必要的部分，这都是很好的。把数据加操作打包成一个概念，再用前缀机制把概念一层一层搭起来，这就是抽象。所谓的抽象，你要做一个模块，不管加到什么名字？一个模块你是用这个class和method，你都需要把这个实现内部的复杂性。

仙！-107老师



画面 00:26:00

(11) IBM, General Purpose Systems Simulator.

(12) Dijkstra, E. W. (1968). Co-operating Sequential Processes. "Programming Languages". pp. 43-112. Academic Press, London.

Took 0.0s

Thinking...

\$ sed -n '1407,1582p' /tmp/sp/hierarchical-program-structures-dahl-hoare.txt (timeout 60s)

... (171 earlier lines, ctrl+o to expand)

enables the programmer to test whether the object referenced by \$r\$ belongs to one of its possible subclasses \$C\$.

7. Concept Hierarchies

Took 0.0s

Thinking...

Working

~/Projects/readspace

775k 47.9K R1.2M CH96.7% \$0.039 8.4%/1.0M (auto)

deepseek-flash • high

需求和架构 (2) · 夸克原版

14

00:26:33

比如说一个选课，或者一门上课。上课这件事儿和系统的其他部分要一定程度上隔离开来。我可以比如说和教室的管理，什么一个教室一个课程，然后他他之间怎么关联，要适当的做这样的分类。



画面 00:27:00

00:27:01

他讲了很多比较有意思的观察。他他说学生写的程序为什么这么烂？是因为一次要同时在脑子里面的东西太多。这个时候你想一想，你们平时哪怕是写OJ题，什么样的题你觉得挑起来最困难？可能不是模拟题，可能是那种需要一点数学性质，比如说动态规划这样的题。你们调这个题，如果你第一次就对了，样例对了，然后数据也对了，可能还挺好的。

00:27:34

如果你一个1 200行的动态规划，你然你你这时候错了，你根本就不知道怎么挑起。这是为什么呢？是因为就这个程序动一个动态规划的程序会比其他程序更难分层。因为你想如果是个模拟，你多多少少还可以把模拟的几个大的部分给分开，封装成一个class。然后这个class你可以单独测试，单独调试。而且你很容易把那个程序的一个中间状态映射到一个你能理解的对吧？你能理解的一个实际物理有物理含义的状态上。



画面 00:28:00

00:28:09

而如果你是调一个像动态规划这种2 300行的动态规划好，那就是灾难。因为它中间是一个数学，是一个纯粹的数学表示。然后这时候你就可以想你有没有可能把这个动态规划的这个层次或者是这个状态改成你能够以我们熟悉的方式去观测它。这事情实际上是可以办到的。就因为我们每一本书上写，比如说写动态文画都是什么DPIJK对吧？

00:28:38

然后你们写的时候也是个DI然后就for I four J 4K然后你只要有一个小的地方弄错了，那整个就全部都错了。而且错有的时候它不会crash，你crash反而好找。你知道这个地方越界了，如果它不crash，它就算出一个错的值，对吧？就你转移的时候弄错了，你就错了。

00:28:57

但实际上在我们谈论这个dp IJK的时候，我们其实是有一个问题的，是有一个优化。一般来讲是一个优化问题对吧？有个优化问题，然后这个优化问题本身是有一个图结构的，也可能是一个图或者别的什么样的结构。然后这个动态规划本质上是在一个状态空间上面做对吧？它有一个无后效行，一个有像无环图，这就是一层抽象。然后在我们写DPHK的时候，其实把这个抽象给扔掉了。你不你虽然我们在思考问题的时候是工作在这个抽象层级上，但是你在写代码的时候把它扔掉了。



画面 00:29:00

二、如果让我“重新发明”这一章，我会怎么想

第 0 步：先承认我们面对的是一个层次问题。

学生写的程序为什么烂？不是不聪明，而是一次量同时在脑子里的东西太多。所以唯一的出路是：把系统切成若干“概念层”，每层只依赖下一层，层与层之间的接口小到可以在一层里讲清楚。这就是 prefix sequence 的动机。

第 1 步：要模拟“并行”，先要有能“暂停/续跑”的程序块。

子程序是主从关系（调用完就回来）；但作业车间里，一个订单执行到一半去排队等机器，之后要从原地继续。这就是协程（coroutine），在 SIMULA 里用 resume / call / detach 表达。

第 2 步：模拟时间和真实计算时间是两码事。

仿真里“等了 3 小时”，可能真实只算了 3 微秒。所以时间必须是一个变量 time，并且规定：只有当所有进程都挂起时，时间才允许前进。这条规则是整个设计的地基。

第 3 步：调度器 = 一个按“该被唤醒的时刻”排序的表。

把所有“将来要醒来的进程”放进一张表 SQS，按唤醒时间递增排序。于是：

- 表的最后一个，就是唤醒时间最小，下一个该跑的进程；
- 它的 time 恰好就是当前仿真时间；
- 每一次循环就是“唤醒最早的那个”——这就是离散事件仿真（discrete event simulation）的核心。

第 4 步：造出四个原语，仿真就活了。

```
---
hold(T)      我睡 T 个仿真时间单位
wait(Q)      我去队列 Q 上排队（离开 SQS）
activate(X)  让 X 立刻跑（同一仿真时刻，先让 X 活动，控制权之后还我）
simulate(start, finish) 总调度 + 时间上限
---
```

00:29:37

所以其实你可以做的是把这个概念状态的概念给放回来。你可以知道D1PIJK它到底代表什么含义。然后你可以把它可视甚至把它可视化出来，甚至可以把这些关联的状态和迁移给打印出来。比如说你可以你可以实现一个比如说cat descendant of IJK，然后它可以返回一个你的状态的迁移，所有的所有可能的迁移。然后你做了这层抽象以后，当然这个可能还是不够好，对吧？



画面 00:30:00

二、如果让我“重新发明”这一章，我会怎么想

第 0 步：先承认我们面对的是一个层次问题。

学生写的程序为什么烂？不是不聪明，而是一次量同时在脑子里的东西太多。所以唯一的出路是：把系统切成若干“概念层”，每层只依赖下一层，层与层之间的接口小到可以在一层里讲清楚。这就是 prefix sequence 的动机。

第 1 步：要模拟“并行”，先要有能“暂停/续跑”的程序块。

子程序是主从关系（调用完就回来）；但作业车间里，一个订单执行到一半去排队等机器，之后要从原地继续。这就是协程（coroutine），在 SIMULA 里用 resume / call / detach 表达。

第 2 步：模拟时间和真实计算时间是两码事。

仿真里“等了 3 小时”，可能真实只算了 3 微秒。所以时间必须是一个变量 time，并且规定：只有当所有进程都挂起时，时间才允许前进。这条规则是整个设计的地基。

第 3 步：调度器 = 一个按“该被唤醒的时刻”排序的表。

把所有“将来要醒来的进程”放进一张表 SQS，按唤醒时间递增排序。于是：

- 表的最后一个，就是唤醒时间最小，下一个该跑的进程；
- 它的 time 恰好就是当前仿真时间；
- 每一次循环就是“唤醒最早的那个”——这就是离散事件仿真（discrete event simulation）的核心。

第 4 步：造出四个原语，仿真就活了。

```
---
hold(T)      我睡 T 个仿真时间单位
wait(Q)      我去队列 Q 上排队（离开 SQS）
activate(X)  让 X 立刻跑（同一仿真时刻，先让 X 活动，控制权之后还我）
simulate(start, finish) 总调度 + 时间上限
---
```

00:30:20

你可以用一个更好的头像，比如说state来替代它。这样你一层一层的做抽象，这个程序你就可以调试了。然后你调完了以后，你可以把这个再改写成等价的更高效的形式。你看这也是一个小小的小小的架格，当然这这是小小的例子。然后你你你各种各样更大的例子的时候，他举了一些像如果你要模拟并行的时候，你就可以创造一个模型，你你你不需要模拟并行，你可以强行的直接模拟，对吧。你说有一个状态机，状态机然后直接模拟，然后你也可以做一个quarting的抽象。这样你只要做出合适的抽象，你hold，wait，activate和simulate它做出一个合适的抽象，你就可以去模拟时间和你直接上手去写一个模拟是啊是很不一样的。



仙一-107老师

画面 00:31:00

二、如果让我“重新发明”这一章，我会怎么想

第 0 步：先承认我们面对的是一个层次问题。
学生写的程序为什么烂？不是不聪明，而是一次量同时在脑子里的东西太多，所以唯一的出路是：把系统切成若干“概念层”，每层只依赖下一层，层与层之间的接口小到可以在一层里讲清楚。这就是 prefix sequence 的动机。

第 1 步：要模拟“并行”，先要有能“暂停/续跑”的程序块。
子程序是主从关系（调用完就回来）；值作业车间里，一个订单执行到一半去排队等机器，之后要从原地继续。这就是协程（coroutine），在 SIMULA 里用 resume / call / detach 表达。

第 2 步：模拟时间和真实计算时间是两码事。
仿真里“等了 3 小时”，可能真实只算了 3 微秒。所以时间必须是一个变量 time，并且规定：只有当所有进程都挂起时，时间才允许前进。这条规则是整个设计的地基。

第 3 步：调度器 = 一个按“该被唤醒的时刻”排序的表。
把所有“将来要醒来的进程”放进一张表 SQS，按唤醒时间递减排序。于是：
- 表的最后一个，就是唤醒时间最小，下一个该跑的进程；
- 它的 time 恰好就是当前仿真时间；
- 每一次循环就是“唤醒最早的那个”——这就是离散事件仿真（discrete event simulation）的核心。

第 4 步：造出四个原语，仿真就活了。

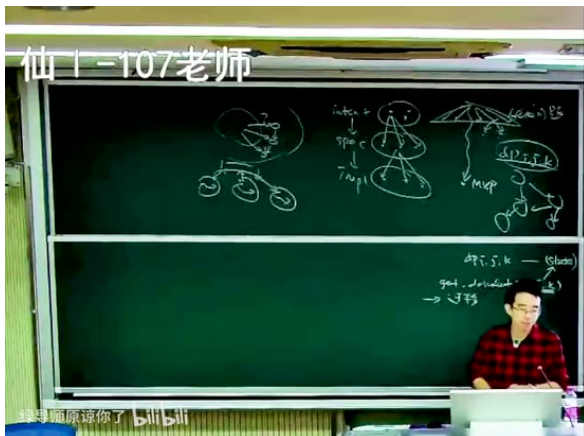
```
...
hold(T)      我睡 T 个仿真时间单位
wait(Q)      我去排队 Q 上排队（离开 SQS）
activate(X)   让 X 立刻跑（同一仿真时刻，先让 X 活动，控制权之后还我）
simulate(start, finish) 总调度 + 时间上限
...
```

00:31:19

这个可能我没有很长就停下。比如说你如果对这个地方不太明白，我相信因为我有这个概念了，我知道他在说什么，你们如果没有你就可以继续的继续的追问，甚至是你跟比如说chat gp开一个语音的赛。因为我最近发现这个feature很好用。大概就是从今年开始，gp t的实时语音模型可以在在后台调用一个更好的思考的模型，并且它可以保持和你对话，这是一个很好的future。

00:31:54

你可以？你可以把论文丢进去，然后你一边问他一些简单，他可以直接立即回你。然后一边你问他一个，比如说你能不能给我想一个打比方，符合现代软件前端的例子，对吧？完全取决于你的提示词，这取决于你的想象力到什么什么程度。我不知道deep能给什么，我完全不知道。



仙一-107老师

画面 00:32:00

wait(Q)，release 是 nm+1（若有人在等 activate 队首）。这和信号量的 P/V 完全同构——作者自己在文末点破了。

三、课堂上值得停下来讨论的几个点

1. “为什么把当前进程放在 SQS 的末尾？”
让学生自己推：如果表按唤醒时间递增排，当前进程应该在哪儿？为什么递减排序最省事？
2. hold(T) 里 if T > 0 和 if time ≥ P.time 的作用是什么？
引导他们发现：T=0 时不该动；时间不能倒退。
3. activate 和 resume 的区别？
activate 走正规的调度通道（会更新 time、进 SQS），resume 是纯逻辑切换。文末警告：“不要对 process 对象直接用 call/resume，那会绕过计时机制。”——这就是封装的实际意义。
4. machine group 到底是数据结构还是同步原语？
答案是：都是。这正是“概念 = 数据 + 操作”的活教材，也是后来 monitor / 信号量 / 各种同步抽象的源头。
5. 对比一下 prefix 与今天的 extends。
vehicle class truck, machine group class Machine Group（重定义 request 加直方图）——这就是继承 + 方法覆写：inspect r when C do ... 就是运行期类型检查。

四、如果你只有 20 分钟，我会怎么讲

提出“并行是假象”→ 协程 → 模拟时间 → SQS 排序 → hold/wait/activate → 跑一个 Lee 算法动画 → 回到“这就是用分层把复杂度吃掉”的主线。

需要的话，我可以继续帮你逐行读 MINISIM 的那段代码，或者现场推演一个 3 进程的 SQS 变化轨迹（把每个时刻的表状态画出来），这对

00:32:21

但是如果你有一个更好的模型的话，它。你看好这个react sc和fiber fiber架构对吧？它找到了这个对应。好，如果我可以再再上升一步，比如说如果我不想谈具体的例子，只是。

00:32:50

抽象的想法有什么？现在最经典的预训练的模型就它没有web部search？它没有web search。如果预训练的模型它就会从脑子里面找到它的那个直觉。但如果你有web search的话，你可以要求gp去调研一百篇1000篇论文，或者从我的那个池子里面的一篇千篇论文里面选一个最好的符合这个的。

仙一-107老师

四、另外两个同样贴切的例子

1. JS 事件循环本身: macrotask / microtask / timer 队列, 就是离散事件仿真调度器, `setTimeout(fn, 100)` 相当于 `hold(100)`, `await` 相当于 `wait(Q)`, `promise` 微任务相当于 `activate()` (同一逻辑时刻立刻排队跑队列者)。
2. Redux-Saga / 生成器中间件: saga 用 generator 描述副作用, `take(action) = wait(Q)` 等某个事件, `put = activate` 唤醒别的 saga, `fork = new process`, 整套 runtime 就是 MINISIM 的协程管理。
3. 并发池 / 限流: machine group 用整数 `nm` 同时表示“空闲资源数”和“排队长度”, `request/release` 就是信号量 P/V, 前端里的 `p-limit`、`AbortController` 管理请求并发, 本质是同一个概念——“资源 + 等待队列 + 释放即唤醒”。

五、课堂讨论点

- 把 `heap.sort(a,b) => b.wake - a.wake` 改成递增排序会怎样? 让学生自己发现“当前任务必须放在队尾”。
- 把 `this.now = Math.max(this.now, task.wake)` 删掉会破坏什么? (时间会倒退。)
- React 用 `expirationTime` 替代 `time`, 这两者在“优先级调度”里是一回事吗?
- `activate` 为什么是“同一逻辑时刻先跑跑完, 再还给调用者”? 对应前端哪个行为?

Recap: React 的 Scheduler (taskQueue/timerQueue 最小堆按到期时间排序) + Fiber (可暂停/恢复的协程式工作循环) 几乎是 Dahl & Hoare 的 MINISIM 的现代重演——同样用“逻辑时间 + 排序任务表 + 协程让出/恢复”在单线程上造出并行假象, 而 machine group 的“资源 + 队列 + request/release”则对应前端的并发池与信号量: 这正好说明第七章的抽象不是历史, 而是今天前端运行时设计的通用骨架。

如果我不想谈具体的例子, 只是这个软件抽象的想法, 有什么现在最经典的案例?

Working

~/Projects/readspace
777k 416k R1.4M CH99.8% \$0.050 9.2%/1.0M (auto)

deepseek-flash + high

00:33:16

他找了一些有意思的unix关系数据库, interface trade protocol TCPIP, 都是告诉你要做一个好的抽象, 很有意思。这几个例子我今天上课都会讲, 所以某种程度上我们还是达成了一定的一致。对, 就是我觉得可能这门课我讲讲什么东西都是不够的, 因为我在这一门课上我没有办法把那一千篇对吧? 比如说我刚才讲 system software, 你读PhD的话, 在我这里读的话, 你得把它把它多多少少都读了。这个还是一个很恐怖的事情。然后但是你们得到启发是读书可能也许没有那么的困难。然后我就在找这些的时候, 我就在在准备今天的课的时候, 我就又去找了一些有点老的材料。那刚才那是1972年, 然后就想如果要到19, 比如说80年、1990年的时候, 我们又怎么来看这件事儿, 对吧?

仙一-107老师

每个需求点都带来审计的代价
- 换到今天, 是 token 开销 (和模型潜在的错误发挥)

架构设计: **分解 essential complexity: 减少 accidental complexity**

- 好的架构能更容易地实现需求, 承受需求的变更
- 同时收缩 spec/impl space 的可能性

阅读经典 (1970s)

Structured Programming (O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare)

"On our inability to do much"

> ...I have a very small head and I had better learn to live with it and to respect my limitations and give them full credit, rather than to try to ignore them...

[[配一个图]]

- Dijkstra 的 Part I 讨论了 our mental aids: 枚举、归纳、抽象。
- 虽然讨论的案例在今天看都是“小程序”, 但 necklace (pearls) 的比喻就是在分解复杂性
- Part III "Hierarchical Program Structures"
- Small head: "precise thinking is possible only in terms of a small number of elements at a time..."
- SIMULA class/object abstraction

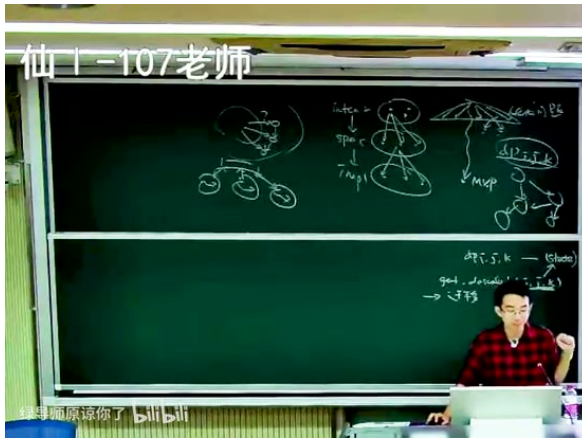
07/lect/index.md 38,0-1 12%

00:34:30

然后我就找到了一个很有意思的talk他是scheme的设计者。对, 他是scheme的设计者, 也是这个Java language specification的合作者。如果你们读过我读过那个Java加Java language和javirtual machine, 他会告诉你语言和讯息是怎么怎么设计的。

00:34:52

然后他在斯酒吧的时候, 就是program language里面的一个讲object oriented的顶级会议。他在98年的时候给了一个keynote。那个时候他已经当然已经是功成名就了, 对吧? 作为那个时候最火的语言之一的设计者, 他他写了一篇文章叫做growing language。我们也来。Article.



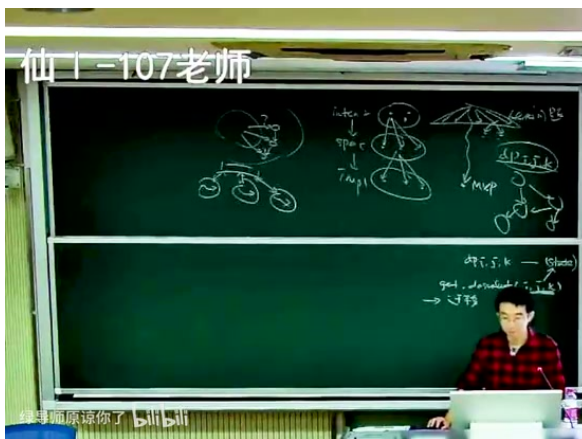
画面 00:35:00

00:35:31

Growing language我让他直接帮我打开嗯，我想要阅读。

00:35:56

是这样，它打开PDFOK。对你你第一次看到它的时候，就是我们作为后人，看到这里有一个PDF。如果你第一次看到他的时候，你会说这是什么呢？Most of the time I do not read my talks，对吧？这个你没理解。如果你直接看到它的时候你没有理解，然后你往后翻到一个这个好像是个什么东西，然后就把它关掉了对吧？



画面 00:36:00

00:36:31

你因为你看到这么多字十几页，你想你你会停下来吗？对吧？停下来吗？所以你应该停下来，这个talk非常有趣，它是一个表演性的演讲。他是写，所以他说I have to read他想要读出来，他全篇都只用单音节英文词，然后遇上所有的双音节以上的词必须当场定义，就跟我说写一个藏头诗或者写一个全是一个音的文章一样，他就做了一个非常炫技的个。所以你看这就这个品味和AI stuff就是不一样的。

阅读经典 (1990s)

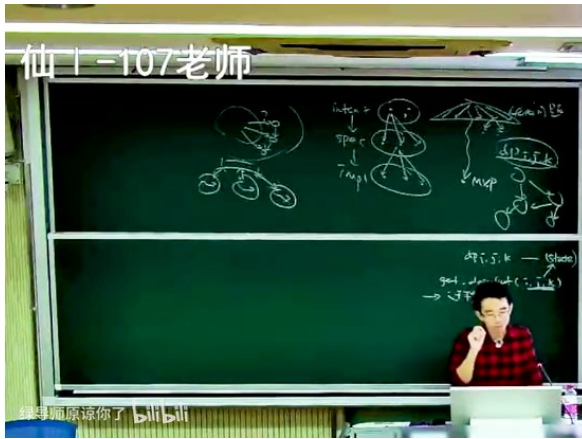
“ One thing the human mind seems to do well is to name things; we have powerful associative memories. Given a name, we can quickly recall some associated thing to mind. (Foreword of SICP, Guy L. Steele Jr)

Guy L. Steele Jr.

- 与 Gerald Jay Sussman 共同设计了 Scheme，也是 SICP 所用语言的创造者之一。
- Common Lisp: The Language 的作者。
- The Java Language Specification 的合著者。

然后我意外地发现了 Growing a language (OOPSLA'98 Keynote)





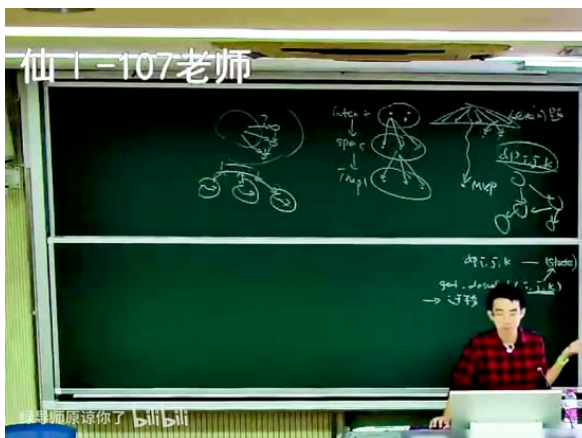
画面 00:37:00

00:37:15

AI stuff写不出这种东西，或者说你需要至少从今天的角度来看，你还需要人类的创造力，你才能想到这件事。而且他用这个方式来讲growing language，就是如果一个语言这个language双关的，你如果要让一个语言能够使用，它刚开始的时候是一个全篇只用单音节英文词，然后他就发现这个语言很难用，当然是说对人类来说很难用。我觉得如果是AI的话，他应该能够做出。

00:37:45

就我觉得今天我可以写一个东西来challenge它，这个文章就全部是用单音节的英文词写的，但是它读起来就完全没有任何问题。他讲的是如果遇到双音阶以上的词，woman machine, vocabulary, programming computer, 必须商场定义。然后这样他就会遇到一个就会遇到那不打开它了，他就会遇到瓶颈，对吧？你你你想要定一个复杂的东西，想要讲清楚任何东西都变得很复杂。



画面 00:38:00

00:38:16

然后他就用这样的—一个很有趣的开场讲了编程programming language里的这个问题。所有的编程语言相对于英语，相对于用户需求都很小，对吧？是的，因为它需编程语言需要有一个forsemantics，它要有形式语义。它要如果这个语言变得非常大，语言里面有这种woman这种概念，那它的形式语义是什么？你定义不出来，而必须有形式语义的东西才能在计算机上执行。所以编程圆都相对来说比较小。然后如果你的编程圆在设计的时候是一个minimal design，就最小。比如说你没用过brain ****对吧？那就完全无法写，你们没有笑，所以。

↓ 反例论证
复数/有理数/区间/Conway 数/向量/矩阵 ...
不可能全部内建 → 应该让用户自己定义，且看起来像原语

↓
Meta: 语言设计本身也是 pattern
「好程序员是词汇的建造者 = 做语言设计」

概述

形式与内容同构。全篇只用单音节英文词，遇到双音节以上的词 (woman, machine, vocabulary, programming, computer...) 必须当场定义。这不是炫技，而是让听众亲身感受在词汇量极小的语言里编程是什么滋味：想说正事之前，必须先花半小时定义术语。

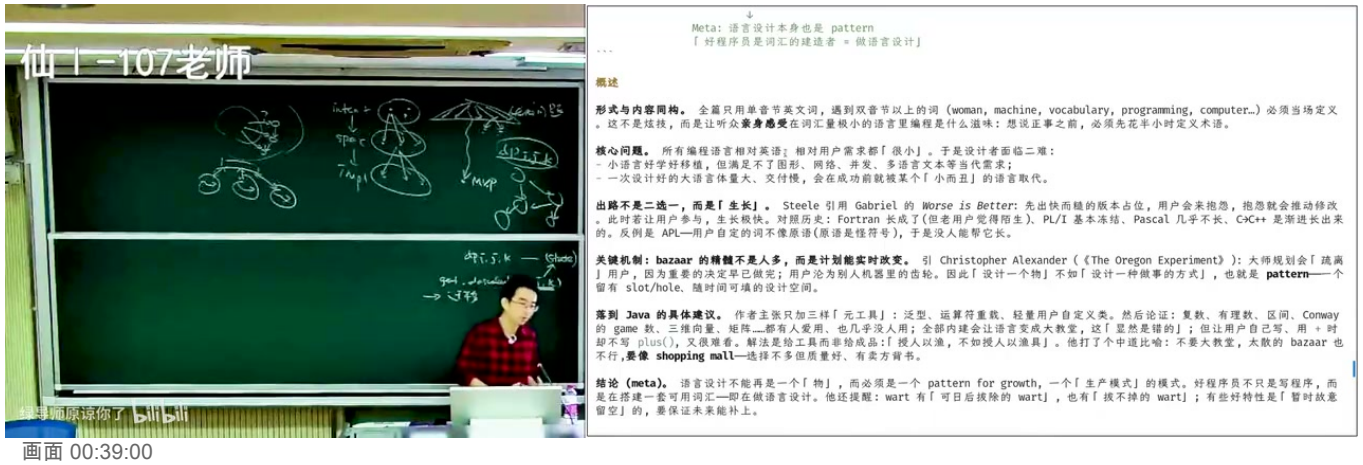
核心问题。所有编程语言相对英语、相对用户需求都「很小」。于是设计者面临二难：
- 小语言好学好移植，但满足不了图形、网络、开发、多语言文本等当代需求；
- 一次设计好的大语言体量庞大、交付慢，会在成功前就被某个「小而丑」的语言取代。

出路不是二选一，而是「生长」。Steele 引用 Gabriel 的 *Worse is Better*: 先出快而糙的版本占位，用户会来抱怨，抱怨就会推动修改。此时若让用户参与，生长极快。对照历史：Fortran 长成了 (但老用户觉得陌生)，PL/I 基本冻结，Pascal 几乎不长，C/C++ 是渐进长出来的。反例是 APL—用户自定的词不像原语 (原语是怪符号)，于是没人能帮它长。

关键机制：bazaar 的精髓不是人多，而是计划能实时改变。引 Christopher Alexander (《The Oregon Experiment》): 大师规划会「疏离」用户，因为重要的决定早已做完；用户沦为别人机器里的齿轮。因此「设计一个物」不如「设计一种做事的方式」，也就是 pattern——个留有 slot/hole、随时间可填的设计空间。

落到 Java 的具体建议。作者主张只加三样「元工具」：泛型、运算符重载、轻量用户自定义类。然后论证：复数、有理数、区间、Conway 的 game 数、三维向量、矩阵... 都有人爱用、也几乎没人用；全部内建会让语言变成大教堂，这「显然是错的」；但让用户自己写、用 + 时却不写 plus()，又很难看。解法是给工具而非给成品：「授人以渔，不如授人以渔具」。他打了个中道比喻：不要大教堂，太散的 bazaar 也不行，要像 shopping mall——选择不多但质量好、有卖方背书。

结论 (meta)。语言设计不能再是一个「物」，而必须是一个 pattern for growth，一个「生产模式」的模式。好程序员不只是写程序，而

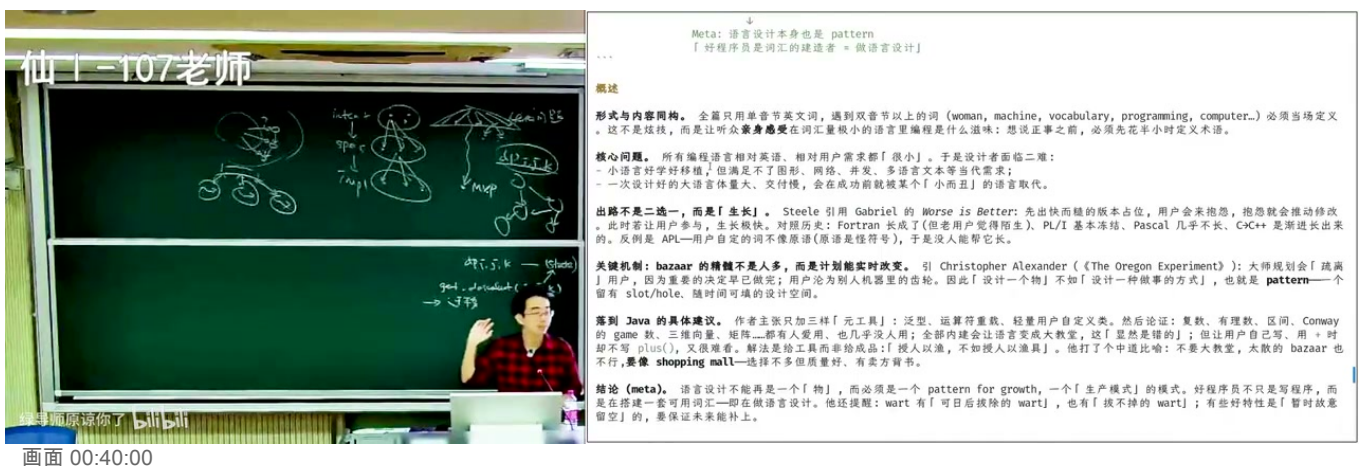


00:39:13

你们想他应该是知道他有预料的，基本上就是个图灵机。就是他把图灵机直接编码成他竟然还要写工具。对他语言基本上就是这样的一串东西，它有一个纸理解成类似图形记有个纸袋，然后你可以往左往右然后做做加法可以判断，对吧？比如说这是一个hello world的程序，对它它最小它满足最小。但当然人类不可读，实际变程语言是当然是不会那么的离谱，还是可以用到的。

00:39:53

但是可以用的意味着比如说C你们C其实是一个设计的非常经典的语言，但是C也带来了很多的设计上的问题。比如说它就没有运算服的重载，对吧？它没有运算符重载。你在写表达这种，比如说你要表达一个往队列里面放东西，如果你们写戏写多的话，就知道这是一件多么恐怖的事情。然后你再看那个C里面有很多比如说这种东西，对吧？Quick shot, 你们可能学过对吧？因为学过C因就用设计的非常糟糕的api对，它有一个的compare的指针，然后它需要这个签名竟然是constboy的，然后anyway这个呃到一个数组里面还需要这个n member b和size两个参数对吧，就C没有这个没有这样的能力让你自定义比较好的语言抽象，所以它的表达任何东西的时候都会遇到困难。你看这两件事情在现实意义上，就他用单音节词去需要定义双音节词的时候，就遇到了困难，对吧？



仙一-107老师



画面 00:41:00

↓

Meta: 语言设计本身也是 pattern
「好程序员是词汇的建造者 = 做语言设计」

...

概述

形式与内容同构。金属只用单音节英文词，遇到双音节以上的词 (woman, machine, vocabulary, programming, computer...) 必须当场定义。这不是炫技，而是让听众亲身感受在词汇量极小的语言里编程是什么滋味：想正事之前，必须先花半小时定义术语。

核心问题。所有编程语言相对英语、相对用户需求都「很小」。于是设计者面临二难：

- 小语言好学移植，但满足不了图形、网络、并发、多语言文本等当代需求；
- 一次设计好的大语言体积量大、交付慢，会在成功前就被某个「小而丑」的语言取代。

出路不是二选一，而是「生长」。Steele 引用 Gabriel 的 *Worse is Better*：先出快而糙的版本占位，用户会来抱怨，抱怨就会推动修改。此时若让用户参与，生长极快。对照历史：Fortran 长成了 (但老用户觉得陌生)、PL/I 基本冻结、Pascal 几乎不长、C/C++ 是渐进长出来的。反例是 APL—用户自定的词不像原语 (原语是怪符号)，于是没人能帮它长。

关键机制：bazaar 的精髓不是人多，而是计划能实时改变。引 Christopher Alexander (《The Oregon Experiment》)：大师规划会「疏离」用户，因为重要的决定早已做完；用户沦为别人机器里的齿轮。因此「设计一个物」不如「设计一种做事的方式」，也就是 pattern——个留有 slot/hole、随时可填的设计空间。

落到 Java 的具体建议。作者主张只加三样「元工具」：泛型、运算符重载、轻量用户自定义类。然后论证：复数、有理数、区间、Conway 的 game 数、三维向量、矩阵... 都有人爱用、也几乎没人用；全部内建会让语言变成大教堂，这「显然是错的」；但让用户自己写、用 + 时却不写 plus()，又很难看。解法是给工具而非给成品：「授人以渔，不如授人以渔具」。他打了个中道比喻：不要大教堂，太败的 bazaar 也不行，要像 shopping mall—选择不多但质量好、有卖方背书。

结论 (meta)。语言设计不能再是一个「物」，而必须是一个 pattern for growth，一个「生产模式」的模式。好程序员不只是写程序，而是在搭建一套可用词汇—即在做语言设计。他还提醒：wart 有「可日后拔除的 wart」，也有「拔不掉的 wart」；有些好特性是「暂时故意留空」的，要保证未来能补上。

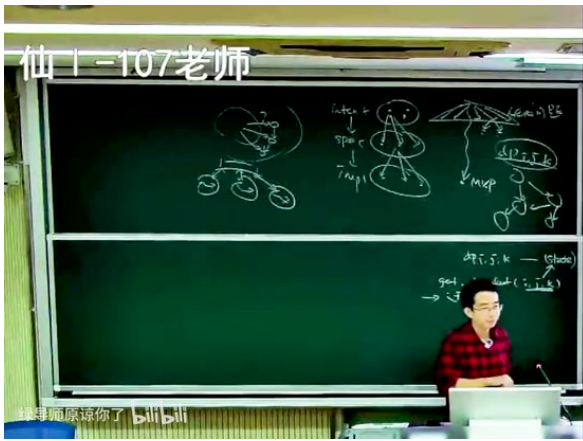
00:41:11

小的语言好移植就好好用，但是满足不了各种各样膨胀的需求。但是一旦设计好，一个大语言的体量大，交付慢，就会在成功前被每一个某一个小而丑的语言取代。所以你看C就是典型非常典型的成功者，他在有些方面很成功，对吧？

00:41:31

我们喜欢C很大程度上是因为它直接和机器是一一对应的，它没有额外的抽象，所以这这就是一个可以移植到几乎任何地方的编程语言。然后单是在人力写代码的年代，对我们写编译器是一个非常复杂的复杂的事情。所以在这个意义下，他做了一个编程语言的架构。你看我今天讲的是架构什么样的。就当谈价格的时候，大家都会觉得软件是个价格，我要用什么样的数据库对吧？我要设计什么样的class来表示student，设计什么样的class来表示course，这个叫叫软，但这是软件架构的一部分。实际上你的编程语言里面有什么？这件事情本身也是架构。

仙一-107老师



画面 00:42:00

↓

Meta: 语言设计本身也是 pattern
「好程序员是词汇的建造者 = 做语言设计」

...

概述

形式与内容同构。金属只用单音节英文词，遇到双音节以上的词 (woman, machine, vocabulary, programming, computer...) 必须当场定义。这不是炫技，而是让听众亲身感受在词汇量极小的语言里编程是什么滋味：想正事之前，必须先花半小时定义术语。

核心问题。所有编程语言相对英语、相对用户需求都「很小」。于是设计者面临二难：

- 小语言好学移植，但满足不了图形、网络、并发、多语言文本等当代需求；
- 一次设计好的大语言体积量大、交付慢，会在成功前就被某个「小而丑」的语言取代。

出路不是二选一，而是「生长」。Steele 引用 Gabriel 的 *Worse is Better*：先出快而糙的版本占位，用户会来抱怨，抱怨就会推动修改。此时若让用户参与，生长极快。对照历史：Fortran 长成了 (但老用户觉得陌生)、PL/I 基本冻结、Pascal 几乎不长、C/C++ 是渐进长出来的。反例是 APL—用户自定的词不像原语 (原语是怪符号)，于是没人能帮它长。

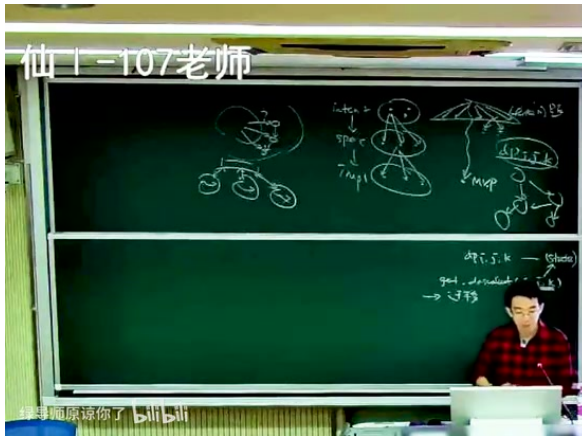
关键机制：bazaar 的精髓不是人多，而是计划能实时改变。引 Christopher Alexander (《The Oregon Experiment》)：大师规划会「疏离」用户，因为重要的决定早已做完；用户沦为别人机器里的齿轮。因此「设计一个物」不如「设计一种做事的方式」，也就是 pattern——个留有 slot/hole、随时可填的设计空间。

落到 Java 的具体建议。作者主张只加三样「元工具」：泛型、运算符重载、轻量用户自定义类。然后论证：复数、有理数、区间、Conway 的 game 数、三维向量、矩阵... 都有人爱用、也几乎没人用；全部内建会让语言变成大教堂，这「显然是错的」；但让用户自己写、用 + 时却不写 plus()，又很难看。解法是给工具而非给成品：「授人以渔，不如授人以渔具」。他打了个中道比喻：不要大教堂，太败的 bazaar 也不行，要像 shopping mall—选择不多但质量好、有卖方背书。

结论 (meta)。语言设计不能再是一个「物」，而必须是一个 pattern for growth，一个「生产模式」的模式。好程序员不只是写程序，而是在搭建一套可用词汇—即在做语言设计。他还提醒：wart 有「可日后拔除的 wart」，也有「拔不掉的 wart」；有些好特性是「暂时故意留空」的，要保证未来能补上。

00:42:20

我前面讲那个Q2 code的二维码的例思是我就创建了一个小的编程语言，这也是一个架构。所以这个架构提了一个非常深刻的问题，说这个出路不是二选一，而是生长。也就是说你的这和其实软件的需求又是对吧？我前面上第一次课时我就讲一个好的软件，它能够或者是软件的设计，它能够抵抗需求的变更。当需求变化，世界发生变化的时候，你一定程度上你在现在可以前瞻性的看到一点，这个未来会发生什么。如果你只是针对现在设计，你可能没有考虑到这个软件应该怎么样的生长。它有可能就会陷入一个一个技术债的泥潭。你需要有这个生长的能力的架构是是最好的。



画面 00:43:00

00:43:16

然后他举了一些例子，比如说一些老的语言，这些语言你们应该都没有用过的，我还用过Bascal，然后C和从C到C加加是一个长出来，但我觉得C加加长得并不好，对吧？从你看从C到C加加，比如说模板，模板是一个非常非常重要的架构。但是在你看到为了实现各种各样的功能，在C加加11、14、十七之前，这个模板被滥用到了，对吧？就boss里面那些模板没有人能看得懂，大家不知道那是对于尤其是对于同学来说，大家都不知道那是什么，你需要花很长时间才才能理解他们对吧？他模板是一个很好的机制，但是长出了一些比较奇怪的奇怪的东西。所以他给Java你看他给Java提供了一些一些建议，他主张只加三样这个原工具范型很重要，没有范型你构造抽象的时候，你是得像C那样对吧？或者早期的Java那样，到处都有强制类型转换，然后运算服重载。好像Java到至今都没有加入轻量用户资定一类，然后这个现在是有。对。他讲了这样的一个架构的故事，那我们也可以问。



画面 00:44:00

00:44:56

我们每一个同学都有自己的偏好，然后你们每一个同学问出来的问题就是不一样的。然后不一样的时候就会有不一样的结果。这场演讲是一条关于growability的架构哲学。呃。

概述

形式与内容同构。 全篇只用单音节英文词，遇到双音节以上的词 (woman, machine, vocabulary, programming, computer...) 必须当场定义。这不是炫技，而是让听众亲身感受在词汇量极小的语言里编程是什么滋味：想说正事之前，必须先花半小时定义术语。

核心问题。 所有编程语言相对英语、相对用户需求都「很小」。于是设计者面临二难：

- 小语言好学好移植，但满足不了图形、网络、并发、多语言文本等当代需求；
- 一次设计好的大语言体量重、交付慢，会在成功前就被某个「小而丑」的语言取代。

出路不是二选一，而是「生长」。 Steele 引用 Gabriel 的 *Worse is Better*：先出快而糙的版本占位，用户会来抱怨，抱怨就会推动修改。此时若让用户参与，生长极快。对照历史：Fortran 长成了 (但老用户觉得陌生)、PL/I 基本冻结、Pascal 几乎不长、C/C++ 是渐进长出来的。反例是 APL—用户自定的词不像原语 (原语是怪符号)，于是没人能帮它长。

关键机制：bazaar 的精髓不是人多，而是计划能实时改变。 引 Christopher Alexander (《The Oregon Experiment》)：大师规划会「疏离」用户，因为重要的决定早已做完；用户沦为别人机器里的齿轮。因此「设计一个物」不如「设计一种做事的方式」，也就是 **pattern**——一个留有 slot/hole、随时间可填的设计空间。

落到 Java 的具体建议。 作者主张只加三样「元工具」：泛型、运算符重载、轻量用户自定义类。然后论证：复数、有理数、区间、 Conway 的 game 数、三维向量、矩阵... 都有人爱用、也几乎没人用；全部内建会让语言变成大教堂，这「显然是错的」；但让用户自己写、用 + 时却不写 plus()，又很难看。解法是给工具而非给成品：「授人以渔，不如授人以渔具」。他打了个中道比喻：不要大教堂，太散的 bazaar 也不行，要像 shopping mall——选择不多但质量好、有卖方背书。

结论 (meta)。 语言设计不能再是一个「物」，而必须是一个 pattern for growth，一个「生产模式」的模式。好程序员不只是写程序，而是在搭建一套可用词汇—即在做语言设计。他还提醒：wart 有「可日后拔除的 wart」，也有「拔不掉的 wart」；有些好特性是「暂时故意留空」的，要保证未来能补上。

结尾的人文意味。 他自述坚持单音节词逼自己想清楚每一个措辞，呼应 Strunk & White 的「能短则短」；最后呼吁掌权者「定义清楚术语、用单音节说话」—因为这样说难以含糊其辞，只能把话说到点子上。

这几点里，最容易在讨论中被追问的是：「加泛型和运算符重载就能让用户自造数值类型」这个论证，在 1998 年的 Java 上成立吗？（实际

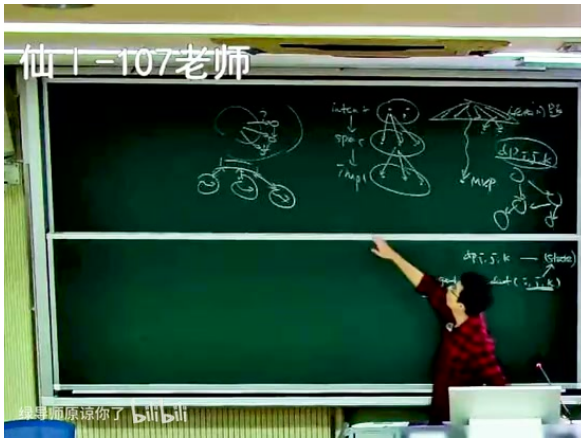
画面 00:45:00

扩展必须是一等公民，对吧？扩展必须是一等功鸣，而且扩展可以之间可以组合，这就是一个可生长的架构。它不是说好我有一个需求清单，然后我把这个需求清单全部都实现了，这叫正确。我们做OJ有一个就有一点，从你们进大学开始？疯狂的做各种各样的OJ题，他就把你们训练成了解题的机器。因为做OJ是不需要任何扩展的，你这个题对了，你就这个代码是不需要维护？你用后即扔的代码，你就会有一种不自觉的像这种用后即扔去走的倾向。好，然后就把你们在这个方面的思考变少了，然后某种程度上这门课会拉回来一点，对吧？

画面 00:46:00

扩展必须是一等功名给原语和组合机制，而不是用力清单，我们可以照着需求清单去写写实现。但是如果我们把这个需求分解成跳出问题看问题的几个概念，然后这个概念又可以变成具体的，比如说领域里面的东西好，然后可以可以组合，那这可能就是一个对的设计。好，我就不继续看了。你们最开始的时候，比如说你们刚开始大二大三的时候，读这些东西肯定是非常慢的。

因为你因为刚才我说，我们是从你已知的知识开始往外推。我现在读论文就很快，是因为我的已知的知识相对来说比较多，而且AI可以帮我整理。那你在建立一个比较好的computer science的核心知识之前，你都会经历一个比较痛苦的过程，就是你不理解为什么。好消息是你可以都跟AI来询问这个啊各种各样的事情。对，好，所以我觉得我有个comment就是有太多的宝藏被埋在这个快速发展的时代里面了。我们我就就大家同学们到了进组了，大家一定是我们最近又发表了一篇什么of art的文章，师兄又发表一篇self的文章。然后我觉得某种程度上，大家的品味就被这些海量的人类slo和AI I slo都污染了。就比如说CDN上面的东西，防止自学的教科书。



画面 00:47:00

00:47:56

然后就这里看到我今天举的两个只是非常随意的例子。可能这些经典就有成百上千这样。我觉得就在在我到我读书的时候，从我比如说本科高年级到我博士毕业的这段时间里面，我我我可能我是不可能把他们都读完的，所以我只读了他们其中的一部分，我算是读书读的比较多的人，也只是很少的一部分。然后有一些自己的想法。但是在今天你们每一个人都可以？每一个人都可以快速的，而且是原汁原味的去了解到里面的作者想要传达的原始的想法。所以我就非常期待一个大学毁灭计划，对吧？

阅读经典 (1990s)

“One thing the human mind seems to do well is to name things; we have powerful associative memories. Given a name, we can quickly recall some associated thing to mind. (Foreword of SICP, Guy L. Steele Jr.)

Guy L. Steele Jr.

- 与 Gerald Jay Sussman 共同设计了 Scheme，也是 SICP 所用语言的创造者之一。
- Common Lisp: The Language 的作者。
- The Java Language Specification 的合著者。

然后我意外地发现了 Growing a language (OOPSLA'98 Keynote)



画面 00:48:00

00:48:45

我们我们有没有可能从就把比如说大一的第一门课programming language就把知识点全拆散了。我们我们不用一本什么，甚至是这个教科书，可能是你们上课的这个老师写的教科书。我们不用这个教科书，我们把这个原始的知识全部都break down，对吧？把1970年、1980年、1990年那些经典的东西全部都拆开来，然后重新串联起来为你生成一个教材对吧？我觉得这个是一个挺好挺好玩的挺好玩的事情。好，这几个是架构的吧？我从growing a language，他说一个能够生长的架构才是最好的架构。

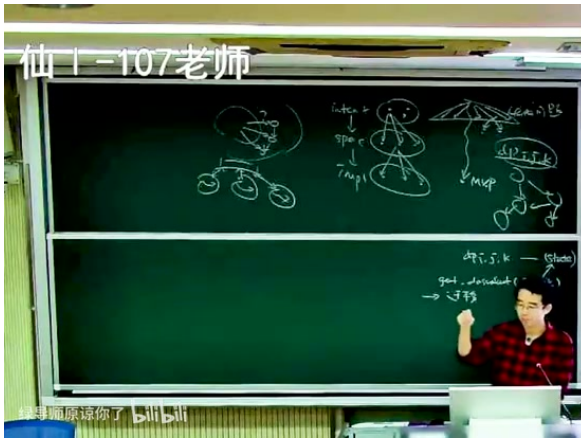
阅读经典，建立品味

Comment: 有太多宝藏被“埋在”这个快速发展的时代了

- 海量人类产物 (CSDN、防自学教科书.....)、AI slop 在疯狂污染大家的品味
- 阅读经典是成长必备的道路
- 我真的很期待一个“大学毁灭计划”
 - 所有学校的改进空间都实在太大了

把经典读进自己的设计

- 拿自己的系统追问：作者把哪些判断交还给了使用者？这些选择，需要什么共同规则才能彼此兼容？



画面 00:49:00

00:49:31

那这个架构设计的空间到底有哪些经典的案例呢？第一个当然就是大名鼎鼎的unix philosophy，三条原则，每一个工具do one thing do well，然后write programs work together，write programs handle text streams？第三条其实是一个非常精髓的设计。因为文本接口统一了人和程序程序和程序的接口。人如果只是binary的字节流的话，对人来说是很不友好的。所以你们在用数据库的时候，你们都知道数据库好，对吧？但是用数据库的时候，你总是觉得隔了一层杀。因为就是你每一次查询任何东西都需要一个seequery，然后这个seequery对你来说是indirect的，而你只是想要一个立即的东西，你人不是很适应在做任何事的时候都要加一层indirection。

阅读经典，建立品味

Comment: 有太多宝藏被“埋在”这个快速发展的时代了

- 海量人类产物 (CSDN、防自学教科书.....), AI slop 在疯狂污染大家的品味
- 阅读经典是成长必备的道路
- 我真的很期待一个“大学毁灭计划”
 - 所有学校的改进空间都实在太大了

把经典读进自己的设计

- 拿自己的系统追问：作者把哪些判断交还给了使用者？这些选择，需要什么共同规则才能彼此兼容？



画面 00:50:00

00:50:24

当然模型可以对吧？比如说你可以让模型每说一句话以后都说一个喵，对吧？他能遵循指令。但如果是人来遵循这个指令的时候，人就会觉得非常非常痛苦了。而这个想法其实在1964年的时候就已经发现了。

00:50:40

然后我们可以同样是原文？这时候你就看到你有一个AI I可以帮你收集资料？一个agent能帮你收集资料，后并且能够帮你总结原文的时候，这就是原文这个四个要点，第一个像接花园水管一样连接程序到管道。

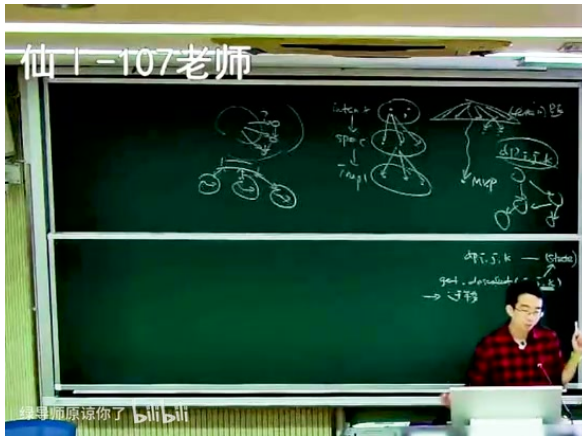
设计解空间：UNIX Philosophy

三条原则

- Do one thing and do it well.
- Write programs to work together.
- Write programs to handle text streams, because that is a universal interface.

精髓是第三条

- 文本接口统一了人-程序、程序-程序的接口，使用者可以
 - 自由组合程序
 - 随时检查程序的中间结果
- McIlroy 在 1964 年就提出：像连接花园水管一样连接程序。原始备忘录



画面 00:51:00

00:51:01

在应该是1983年的时候引入unix但其实你看这一条在那个时候就已经有了这个想法了。然后你看它后面有很多其他的关于操作系统设计的非常经典的想法对吧？动态的动态连接，这个酷的索引，然后你看到这些对今天的这个呃系统设计都是有非常深远的深远的影响对吧？

00:51:33

Unix然后unix就是依套一套unix世界里面的语言。当然你们也可能看过那个叫unix hater guide，就各种吐槽unix里面的这种设计的设计的不好。但是就是一套统一的标准，所有的命令行工具都遵循大致一样的命令行参数，这样人就可以理解了，人很好理解。然后而且他们输出的这个格式也尽可能的保持一致。因为当有几大工具，比如说赛德AWK这种工具奠定了技术以后，程序的输出也开始向这几个工具友好的方向，同时对人也对工具友好的方向来继续以及这个退出状态？Astatus它可以在程序出问题的时候及时的告诉另外一个程序。

《Prophetic Petroglyphs》(Dennis Ritchie, 关于 Doug McIlroy) 页面摘要

背景

- Ritchie 办公室墙上贴着一张泛黄纸页，是 1964 年 Bell Labs 内部备忘录的第 10 页，作者 Doug McIlroy。
- 当时 Bell Labs 用 IBM 7090/7094 批处理，自研 OS 叫 BESYS；软件上有 BLOOM, Snobol 等语言工作，McIlroy 专长是宏处理。
- 1964 也正是 Multics 项目起步之时，这些想法影响了 Multics 设计，也促使 Bell Labs 参与其中。

原页四条要点

1. 像接花园水管一样连接程序——需要时拧上另一段来加工数据。IO 也应如此。
2. 加载器应支持 link-loading 和受控建立(controlled establishment)。
3. 库的检查机制应支持通用索引、责任(responsibility)、世代(generations)、数据通路切换。
4. 应当能拿到私有的系统组件（所有例程都是系统组件），便于胡乱折腾(bugging around)。

签名: M. D. McIlroy, 1964 年 10 月 11 日。

Ritchie 的解读

- 第 1 点最显著: 它是后来 Unix 管道(pipe)记法的历史思想源头; McIlroy 在访谈中也说，他几乎动用了管理权力去推动 pipes 落地。这个“花园水管”比喻最终带来了最大影响。
- 第 2 点若脱离上下文较难懂，可读作主动态链接(dynamic linking)—Multics 首创的机制。
- 第 3 点大概是至今仍困扰 McIlroy 的问题: 源码管理、注册表、WWW 搜索引擎等机制都有，但“索引并找到相关信息”这件事今天反而更难，连自己的硬盘都难，更别说整个 WWW。
- 第 4 点在某种程度上已被解决(如 Multics 的 dynamic linking + 搜索路径及其后代)，或可勉强联系到自由软件运动，但 Ritchie 猜他想的更实际: “如何拿一个看似铁板一块的程序，去测试我刚写的、还想验证的 sin 例程?”

一句话: 这页 1964 年的“预言石刻”，核心就是那句“像接水管一样拼程序”，它直接催生了 Unix 管道。

Recap: 通过这一页 1964 年 McIlroy 备忘录，我们追溯了 Unix 管道(pipe)与“组合小工具”哲学的思想源头，也看到动态链接、库检索、私有组件等现代概念的早期雏形—好设计往往是几十年前的直觉被反复验证。



画面 00:52:00

00:52:25

好，就这几个加上管道，这就是一个非常棒的机制。比如说当我们需要比如说你们要设计一个软件，然后这个软件需要一个打开文件的菜单。那这个打开文件的菜单要么就是它自己实现一个，要么就是使用操作系统的。但如果你对这两个都不满意，你有办法吗？没有办法，对吧？

00:52:54

你想一想，学校买了PR对吧？买了那个PR我记得P的就是倒闭全家桶里面那些打开好像是他自己的菜单，因为它可能会提供一些预览这样的功能。然后在那个菜单你说它好用，它好用的时候挺好用，说难用，它不符合符合你使用习惯的时候很难用。然后如果你是一个操作系统的打开文件文档文件的窗口，那也不是那么的好用，对吧？

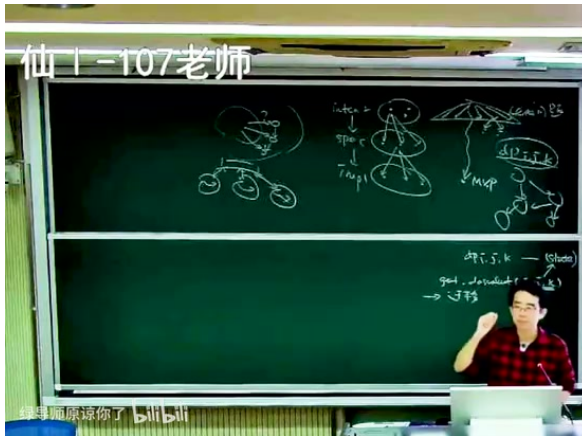
UNIX：组合需要共同约定

POSIX：命令与进程的公共接口

- ④ 参数：短选项、选项参数、操作数：-- 结束选项解析（个别工具有历史例外）。语法定约
- ④ 数据与诊断分开：stdin 输入、stdout 输出、stderr 诊断：| 默认连接前者的标准输出与后者的标准输入。Shell
- ④ 退出状态单独传递：0 通常表示成功，其他值按工具约定解释；例如 grep 的 1 表示未匹配。退出状态

社区实践：让文本成为稳定的数据接口

- ④ 简单记录常用“一行一条”、固定字段：复杂结构用 TSV、JSON 等明确格式。
- ④ 机器模式保持稳定：彩色、进度条、提示语不混入数据流。
- ④ 分隔符也是协议：文件名可以包含换行，传文件列表可用 NUL 分隔。GNU 工具实践
- ④ 排序、字符分类会受 locale 影响：需要可复现规则时可固定 LC_ALL=C。POSIX 环境变量



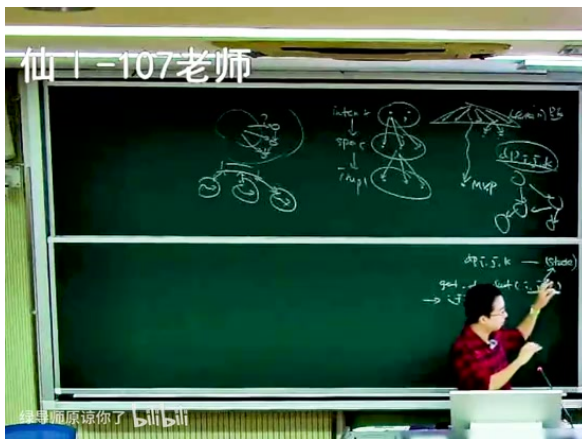
画面 00:53:00

00:53:23

虽然它集成了绝大部分常用的功能，但unix里面这个答案不一样。比如说我现在想要为我的任何一个东西，对吧？比如说我现在用vim或者用e max我们不胜战，对吧？你用任何一个都可以，或者用nano，我有没有nano？好，我有。这个我想给他配一个菜单，我就可以配一个我自己的菜单，然后就得到了一个。

00:53:49

你看这是一个打开文件的菜单，这就是工具的组合性，它这个模块对吧？因为unix当然这是一个偶然可能在系统的设计者也没有想到那么多那么多事情。但比如说我就可以随便随便找一个，比如说第一课的index，我就用nano打开它了。所以看到share这个share的文本接口和编程语言，还有unix的管道文本接口，它是一脉相承的。把它放在一起，你就有一个无限组合和复用的空间，允许你写程序。



画面 00:54:00

00:54:27

所以我上操作系统课的时候，我经常说你们都是程序员。你们在用Linux用unix的那一刻起，你每个人都是程序员。因为你做的每一个命令，它都是一个小程序。然后这个小程序就是负责把各种各样的多关性and do well的工具给组合起来，这种可组合性是非常关键的。到今天你都很难在一个ge程序里面得到这样的组合性。那我觉得这样的ground br东西也是可以应该是可以做的出来的。可以畅想一下，这是uniunix。

00:55:00

然后第二个也是大家熟悉的例子，其实我已经讲过几次了，就是关系数据库对吧。A relation model of data as a basis for protecting user from formmatted data system from potentially disruptive changing in data representation caused by growth in the data bank and changes in traffic。也就是说，如果你用文件系统去管理数据，你就面对一个raw interface。它的好处是unix世界里面的工具可以复用，它的坏处是你你太过自由，以至于它没有这种统一的管理。

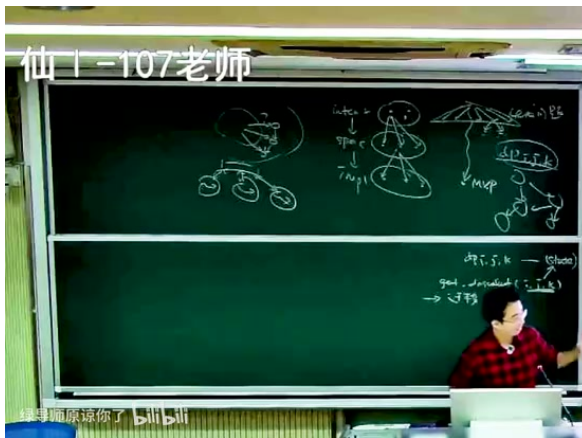
UNIX：使用者创造新的用途

现代工具继续扩展组合的边界

- `jq`: 把JSON中需要的字段提取成文本流，API响应也能进入流水线。
- `fzf`: 从标准输入读取候选项，让人选择，再把结果写回标准输出。
 - `书目JSON` → `jq`提取书名 → `fzf`人工选择: 人的判断也是可组合的一步。
- `xdotool`: 把窗口搜索、键盘和鼠标操作变成命令（适用于X11）。
 - `窗口ID列表` → `fzf`选择 → `xdotool windowactivate`: 组合出自己的窗口切换器。

设计者提供了组合的规则，使用者可以创造设计者没有想到的用途

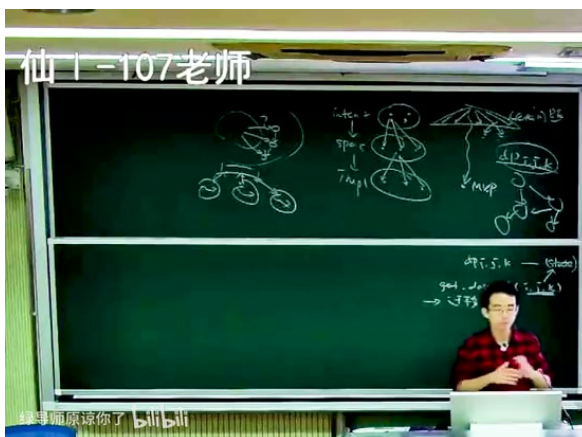
```
assets/head.png
p6/jw-demo/vite.config.ts
p6/jw-demo/tsconfig.node.json
p6/jw-demo/tsconfig.json
p6/jw-demo/package.json
p6/jw-demo/package-lock.json
p6/jw-demo/index.html
p6/jw-demo/README.md
p6/jw-demo/.prettierrc.json
p6/jw-demo/.oxlintc.json
p6/jw-demo/.gitignore
p3/lect/happy-to-learn.webp
p3/lect/gpt6-release.jpg
p3/lect/gpt6-3d.jpg
p6/jw/验收场费.csv
p6/jw/需求跟踪矩阵.csv
p6/jw/需求说明书.html
p6/jw/需求清单.csv
guest/lect/07cs.webp
p1/lect/tar-pits.jpg
p1/lect/spec-gap.png
p1/lect/qrgen.xlsx
p1/lect/liangwenfeng.gif
p1/lect/index.md
p1/lect/ds-harness.jpg
p1/lect/OS2026-02.pdf
p1/lect/CV.pdf
scripts/test_lectcat.py
200/208
```



画面 00:55:00

00:55:35

然后这个关系代数，他做了一个非常了不起的，当然我觉得就是因为我自己做过一段时期的程序合成，然后发现我一旦凡我要做这个和人类世界相关的这程序合成，我们绕不开的就是1970年的这个论文。它其实是对世界建立一个模型。他说我们的世界里面的数据可以用这个二维表，对吧？我相信大家应该是熟悉的，就用好多好多的表和表之间的一些操作来表示。然后你也几乎可以用这个表和表之间的，比如说中云这样的操作来处理任何你所需要的数据管理。而这个数据怎么管，怎么存，它可靠不可靠，全部交给底下的数据库系统的实现。也就是说在这个关系代数的基础上，我们有了结构化的查询语言。



画面 00:56:00

00:56:36

这个语言你看也是一个语言，上面unix share bath也是一个语言，然后这个sicle也是一个语言。所以当你grow一个东西的时候，一个东西可以grow的时候，它多多少少都会带一个语义的形质。它是一个接口，然后这个接口它足够的灵活，所以它就会像一个编程语言，si口有了si口，然后有了查询优化器，然后有了事物以后，那就数据库在任何时候，这个二维表都是当前世界的一个状态。然后这个状态经过一次数据库的transaction会到下一个状态。然后这个数据库还可以有视图，比如说we？

UNIX：使用者创造新的用途

现代工具继续扩展组合的边界

- jq: 把JSON中需要的字段提取成文本流，API响应也能进入流水线。
- fzf: 从标准输入读取候选项，让人选择，再把结果写回标准输出。
 - 书目JSON → jq提取书名 → fzf人工选择：人的判断也是可组合的一步。
- xdotool: 把窗口搜索、键盘和鼠标操作变成命令（适用于X11）。
 - 窗口ID列表 → fzf选择 → xdotool windowactivate: 组合出自己的窗口切换器。

设计者提供了组合的规则，使用者可以创造设计者没有想到的用途

案例二：关系数据库

“ A relation model of data as a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic. (E. F. Codd, 1970)

把物理世界状态投射到关系数据库中

- 分离“数据的理解”与“数据在机器中的组织”
- “谁借了哪本书”保持不变，数据页、索引、访问路径可以改变。Codd的数据独立性

在逻辑关系上构建应用

- SQL、查询优化器、事务实现使CRUD成为标准的软件架构pattern
- 随后发展的关系数据库系统，把“声明要什么”落实为可选择的执行计划。System R (1976)



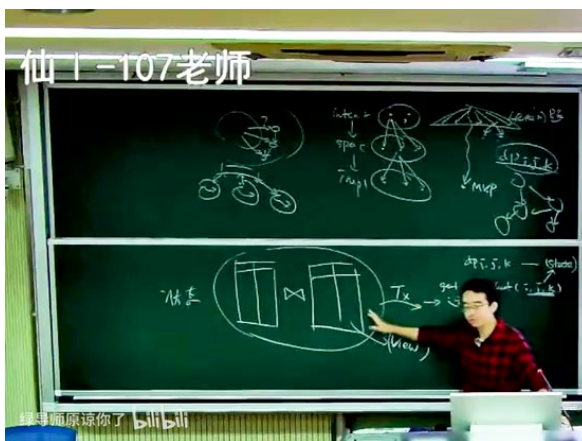
画面 00:57:00

00:57:17

我当前的状态可以用一个查询来创建一个我对这个世界当前世界的理解。而且数据库系统的实现给了我们近乎无限scale的能力，对吧？到今天我们可以的，数据依然可以无限的scale，然后我们就可以用这样的一个信息系统。所以数据库其实带起了整个人类世界的数据库信息化的一波很大的浪潮。

00:57:47

你想图书馆里的书可以信息化，学籍管理你的系统可以信息化，你的财务的信息可以信息化，在线商城的订单可以信息化，所有东西都可以用这个relation database来表示，它是逻辑关系，它表示了实体和实体之间的逻辑关系。只要把这个逻辑关系理清楚，你就可以在现实世界里面实现信息系统。这也是一个非常顶级的？隔离了复杂性的抽象，而且它有无穷的扩展能力。它只规定说我这个世界里面有表database schema，但这个schema是什么？可以由你来扩展，由你来决定为这个应用来来定制。然后第三个案例就是Internet，Internet也是一个非常典型的案例，它解决一个非常简单的问题，我今天为什么可以站在那上课？上课的时候我同时还在推流，然后我推流的时候，我的手机同时还能监看我推流的结果和弹幕，对吧？



画面 00:58:00

案例二：关系数据库

“ A relation model of data as a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic. (E. F. Codd, 1970)

把物理世界状态投射到关系数据库中

- 分离“数据的理解”与“数据在机器中的组织”
- “谁借了哪本书”保持不变，数据页、索引、访问路径可以改变。Codd的数据独立性

在逻辑关系上构建应用

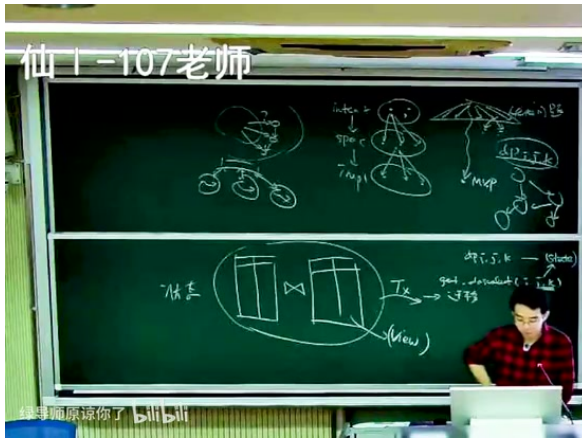
- SQL、查询优化器、事务实现使 CRUD 成为标准的软件架构 pattern
- 随后发展的关系数据库系统，把“声明要什么”落实为可选择的执行计划。System R (1976)

关系数据库：现实、关系与存储

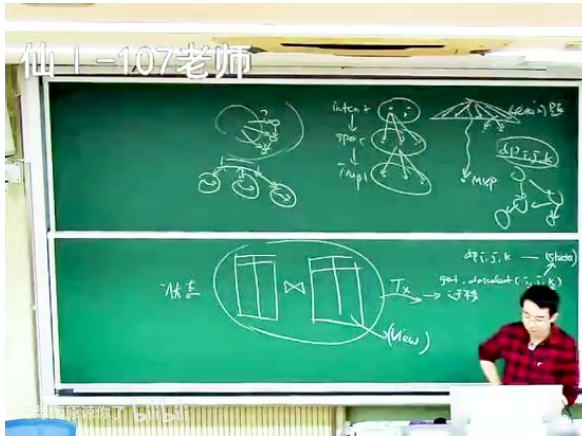
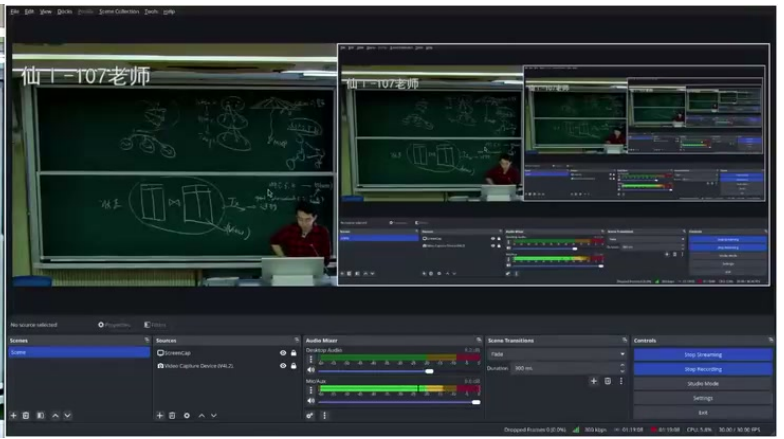
同一份数据，支持不同的问题

- 记录当前借阅：按读者查“借了什么”，按馆藏查“被谁借走”。

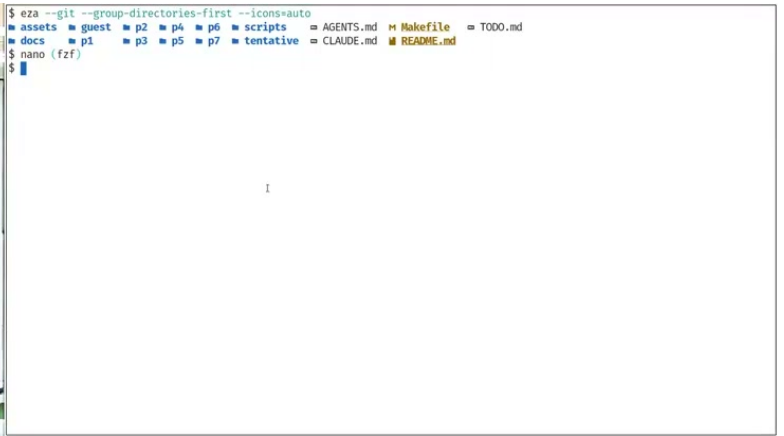




画面 00:58:41



画面 00:58:45



00:58:48

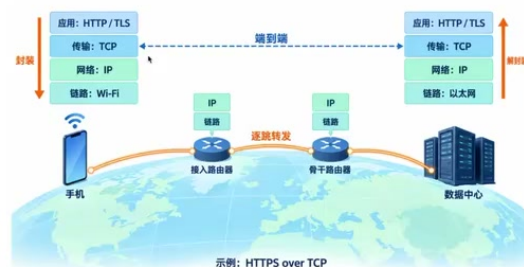
他把整个世界给连起来了，然后这个连起来它也是一个可以成长的架构。他希望的是我要做一个架构，他他负责他能够解决这个系统里面一部分的问题。比如说你当我用TCP的时候，你再也不用担心这个数据传输到底可靠不可靠。我无论什么样的数据流都可以稳定可靠的从一个点送到另外一个点，对吧？从我的手机上和另外世界上另外一头的某一个点连起来都是没有问题的。然后在至于上面跑什么样的应用程序，跑什么样应用程序，我能够支持什么就支持什么，不要做预设。所以你可以读一下这个，我觉得还是这个也是非常非常非常经典对吧？



画面 00:59:00

案例三：The Internet

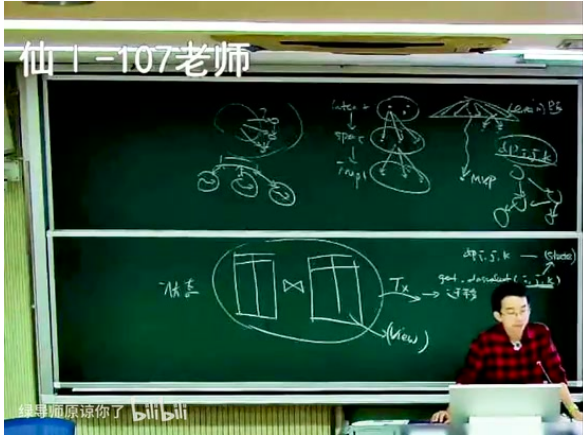
手机与数据中心如何通信：TCP/IP 分层



00:59:40

RFC1958, rc 191958, architecture principle of the Internet, 1996年6月架构的原则。唯一应该永存的原则就是变化本身。这个原则就是说他要承载这个世界的变化。所以他就做一件事情网络networking。他要做一件事情目标就是connectivity。然后在connectivity的基础上,上面就会有很多的应用可以爆发出来,但是他不管对吧?甚至比如说现在有些数据中心甚至还在用tc BIP,虽然现在有更好的吧?如果你要真的要在,比如说要在g pu之间传输数据,那可能需要更好的协议。

仙一-T07老师



read /tmp/rfc1958.txt:40-459

Thinking...

已读完 RFC 1958 - Architectural Principles of the Internet (IAB, 1996年6月, Informational)。

核心立场

Internet 不是“大设计”出来的,而是像不断翻新街道和建筑的城市——不推倒重建,所以这份文档不是教条,而是一份“当前理解快照”。

唯一应该永存的原则,就是变化本身这个原则。 (1. Constant Change)

关键论点

- 1. 架构三要素 (§2)**
 - 目标是 **connectivity** (连通性本身就是回报)
 - 工具是 **IP** (互联网层必须与硬件、硬件寻址解耦)
 - 智能放在 **端到端 (end-to-end)**, 而非网络内部
- 2. End-to-End 论证 (引 Saltzer)**

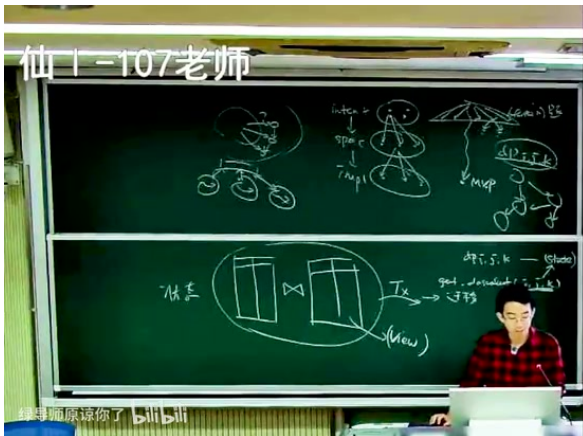
端到端功能只能由两端系统正确实现;网络只管高效传 **datagram**。“其余一切放在边缘”。推论是 **fate-sharing**: 状态只存在于端点,端点崩了状态才故障。网络内部状态必须自愈、最小化、可临时丢失而不中断服务。
- 3. 一般设计原则 (§3, 最实用的一节)**
 - 多样性不可避免,设计必须支持 (速度差 7 个数量级...)
 - 同一件事只选一种做法,能简单就简单: **模块化**
 - 宁重现在就能用的近乎完美的方案,不要完美方案
 - 尽量避免选项和参数,要配就动态协商
 - 发送严格、接收宽容 (Be strict when sending, tolerant when receiving)
 - 避免循环依赖 (如路由不能依赖 DNS)

画面 01:00:00

01:00:34

网络只管高效传data,但是这个传的时候还有很多细节,比如说像提示B点有这种出色控制这种难题,然后还有这种断连连接的管理,对吧?连接的中断什么时候就决定这个连接就中断了。然后这些是非常重要的,它架构里面需要包含的设计,然后其他的基本上就全部都交给别人。好,在这个基础上他做好了这个connectivity这一件事,上面的世界其实变了很多。

仙一-T07老师



Internet: 在持续变化中演化

RFC 1958: Architectural Principles of the Internet (1996)

“ In searching for Internet architectural principles, we must remember that technical change is continuous in the information technology industry.”

做好“数据传输”这一件事

- Telnet → ssh
- FTP/HTTP/... → HTTPS (许多常见用途的迁移)
- SMTP/IMAP 一直到今天还在使用

在共同规则之上,持续更换局部

- RFC 用城市作比喻: 街道与建筑逐步更新,城市仍持续运转。
- 少量合作规则,可以生成庞大、多样、不断演化的技术空间。RFC 1958 §1

画面 01:01:00

01:01:06

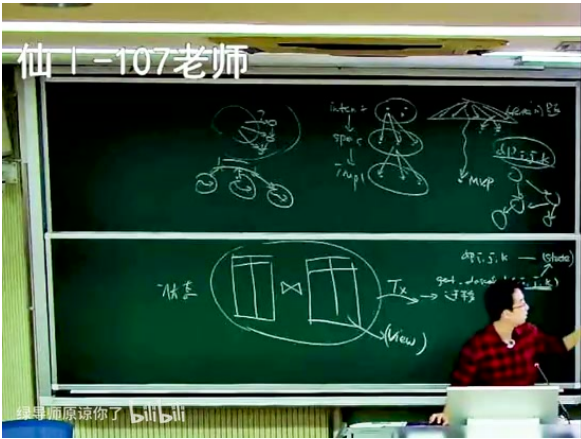
早在早期的时候,我跟大家说过,我读书的时候整还没有现在这个全网HTTPS,我们都是HP上裸奔的。然后在线上裸奔的时候,我们都是直接用tinet去连到南京大学小百合BB去灌水的。但今天你看他那显然是已经安全性等等都达不到我们要的要求。

01:01:28

我们有SHH,然后在我那个时候传出文件时,用的f tp。F tp为什么好呢?Ftp是因为它协议里面本身就支持断点续传,所以那个时候我们一下电影,在校内里面下电影下的非常开心。我因为我下那个时候网络可能不是很稳定,或者反正经常有问题,或者我下到一半的时候,我可能就要我要停下来。但我下一次我可以从这个断点继续传输,在协议里面支持了。但今天他就显得也是因为因为没有安全等等。

01:02:00

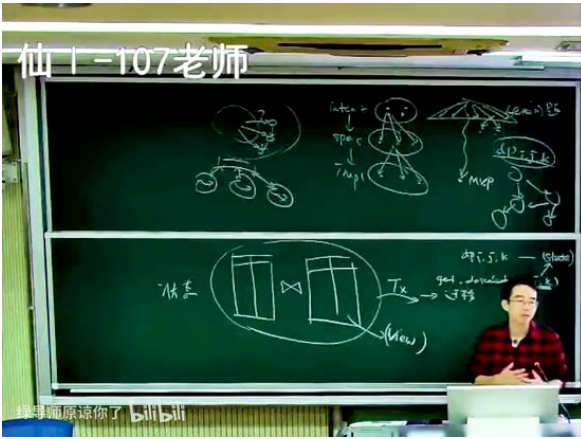
虽然可能有安全的扩展，但是今天做软件变容易了。所以我们可以把这些功能全部实现在HTP的一个接口里。所以HTTPS又变成了一个有点像这个TCBIP的上一层，今天你访问所有的这个mico services，你的所有的你只要出了你的本地的东西和Internet上面的另外一个服务商，全部都是HEPS，然后他有这个re的api的这样的一个规范。当然是建议的，你可以不遵你可以不严格严格遵循它在这个体系上面又长出了很多的东西，在共同的规则之上持续的更换局部。



画面 01:02:00

01:02:42

所以就好像是他建设了一个城市的交通网络，然后这个建筑它就一直都在不断的向前演进。这几个是其实是大家比较熟悉的例子。你们会在就为什么要在你们学大学里面学这些课程，对吧？当然你肯定是要学一些具体的技术。比如说你有一天你真的会出去写写si对，真的有一天你脱网的时候，你真的需要一个si口的时候，你可能你还得自己手写一下，你得有有概念对着手才能写出来。



画面 01:03:00

01:03:15

但另一个方面，你更多的是看到这些沉淀下来的经典，它为什么能成为经典？这些系统当然我讲的是可能它偏软件价格的这一方面，它是怎么样应对变化。你可以体会为什么这样的一个好的设计，它可以支撑起一个生态，应对这些变化。当然我们还是讲软件工程的？这是生成式软件工程，这个呃我们还是讲软件的架构的，我们在在刚才的这种systems里面看到了一些。因为systems是最它它是为了提供支撑应用的通用抽象层，所以它会特它让一半要做的特别好，才能在很长的演化里面存活下来，对吧？

Internet：在持续变化中演化

RFC 1958: Architectural Principles of the Internet (1996)

“ In searching for Internet architectural principles, we must remember that technical change is continuous in the information technology industry.

做好“数据传输”这一件事

- Telnet → ssh
- FTP/HTTP/... → HTTPS ☞ (许多常见用途的迁移)
- SMTP/IMAP 一直到今天还在使用

在共同规则之上，持续更换局部

- ☞ RFC 用城市作比喻：街道与建筑逐步更新，城市仍持续运转。
- ☞ 少量合作规则，可以生成庞大、多样、不断演化的技术空间。RFC 1958 §1

Internet：在持续变化中演化

RFC 1958: Architectural Principles of the Internet (1996)

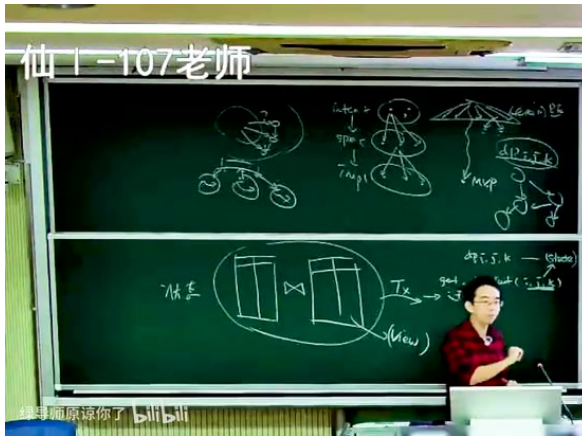
“ In searching for Internet architectural principles, we must remember that technical change is continuous in the information technology industry.

做好“数据传输”这一件事

- Telnet → ssh
- FTP/HTTP/... → HTTPS ☞ (许多常见用途的迁移)
- SMTP/IMAP 一直到今天还在使用

在共同规则之上，持续更换局部

- ☞ RFC 用城市作比喻：街道与建筑逐步更新，城市仍持续运转。
- ☞ 少量合作规则，可以生成庞大、多样、不断演化的技术空间。RFC 1958 §1



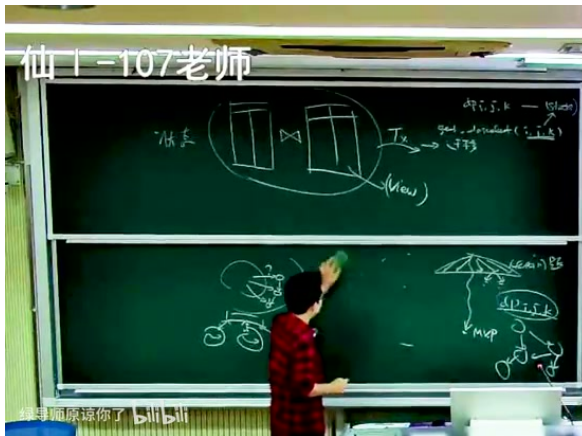
画面 01:04:00

01:04:02

从ib m360360是一个非常经典的系统，然后到unix面向对象的编程语言，Internet android android我应该会专门有一次课来讲。然后到比如说云原生里面的这种，那我们有没有面向客户需求的软件架构呢？对吧？软件架构当系统这一层都打好了，地基打好了以后，我们有了操作系统，在操作系统上面有了数据库，然后在操作系统上面有了Internet。这个舞台其实就已经为人类搭建好了各种各样的业务逻辑系统、教务系统、图书管理系统就全部都可以上线了，对吧？

01:04:44

那我们在这个已经打好的抽象层的基础上，我们能做什么呢？很自然的想法，如果你你你今天想象你回到1998年或者回到2000年，你要设计为我们的学校或者为企业设计软件系统。你手上有的英特瓦数据库操作系统，你最直接的想法就是你为每一个页面或者为每一个功能都做成是一个siq的transaction对吧？因为数据库已经完成了物理世界过程在信息世界里面的投影，对吧？它就是一个好多表格的这样的一个分解。



画面 01:05:00

01:05:28

我们刚才说是由数据库的事物驱动这个状态的变化，而我们可以把需求这时候你就能对上，你就可以可以理解为什么那些软件工程的书你看不明白。那些软件工程书他要这样写。他要把这个需求软件工程做的事，或者说如果你要构建业务系统做的事，就是把需求投射成不同的对吧？也就是一个分解不同的需求点，然后每个需求点变成一个数据库的transaction，然后以及我可以为这个当前的状态提供各种各样的数据库的视图，数据库视图也是一个查询。比如说我想要看有哪些学生选择了我的这一门生成式软件工程课程，那我就写一个select对吧？这样把把这个数据库的视图的某种程度上，你可以理解成是一个一个切片拿出来，然后这个切片就可以显示在某一个地方，这样我们就得到了一个最最经典的应用程序的架构。

Systems: 每一轮浪潮的新抽象

为通用应用搭建抽象层

- 刚才三个都是“Systems”，是为了支撑通用应用做的抽象层



史料: IBM · UNIX 原作者 · Alan Kay · CERN · Android · Kubernetes

1/19

如果是面向客户需求的软件呢？

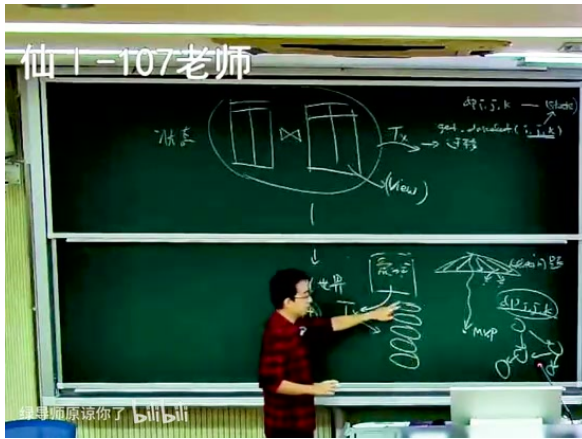
操作系统、CRUD 数据库、Internet 连接了服务器和世界

- 最直接的想法：每个页面就是一个 SQL Transaction
 - 就可以支持在线商城、社交网络、教务系统.....
 - 选课请求：读课程 → 检查能否选 → 写选课记录 → 返回页面

业务代码应该放在哪里？

- 学生选课页、管理员补选课，都要遵守课程容量限制
- 把规则复制到每个页面，改一次规则就得找遍所有入口

2/19



画面 01:06:00

01:06:38

然后这个架构一定程度上其实是可以就在这个需求点，相对就每一个需求点都是一个数据库事物查询的时候，这个价格其实是非常好的这价格其实是非常好的。然后直接有一波应用的浪潮。在包括这有好几个因素，包括这个操作系统CUD数据库和Internet。

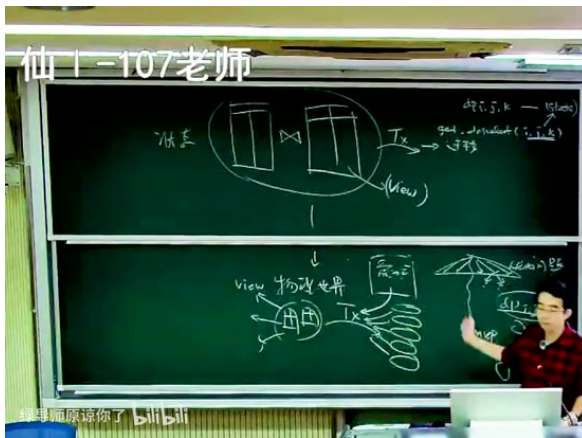
如果是面向客户需求的软件呢？

操作系统、CRUD 数据库、Internet 连接了服务器和世界

- 最直接的想法：每个页面就是一个 SQL Transaction
 - 就可以支持在线商城、社交网络、教务系统.....
 - 选课请求：读课程 → 检查能否选 → 写选课记录 → 返回页面

业务代码应该放在哪里？

- 学生选课页、管理员补选页，都要遵守课程容量限制
- 把规则复制到每个页面，改一次规则就得找遍所有入口



画面 01:07:00

01:07:06

以及刚才我讲到那个呃欧star酒吧的keynote里面，他讲Java语言的设计。那个时候1998年Java如日中天的时候，Java在允许我们直接在HTML里面写代码。当然这个不是一个很新的想法。比如说今天我们你们你们如果要对这这也是一个很好的架构，叫模板语言。你们有没有写过自己写一个AI应用，然后你需要自己定制一个prompt。我不知道你们有没有写过这个需求，然后你们怎么拼prompt的？如果你们想要，你想要prompt的你们会怎么样？是prompt的等于A加B加C这样拼，你可以这样拼，然后加上一个sorry，写好多prompt，然后再加上什么D再加上你们会这样写prompt的吗？

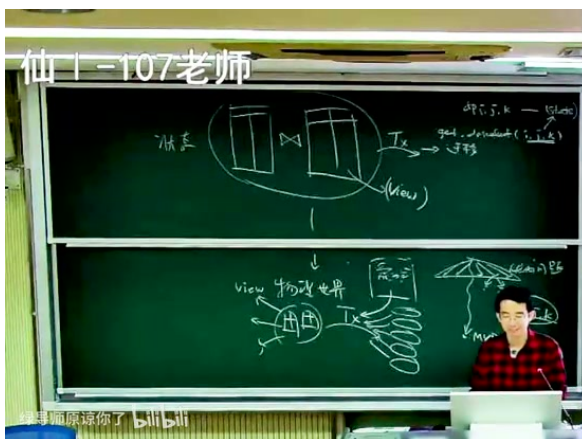
Java Web：直接在 HTML 里写代码

Java Web 浪潮 (1990s 后期)

- Java Virtual Machine: Write once, configure everywhere 😊
- 自动内存管理，程序员救星

页面模板：让 HTML 留在 HTML 里

- PHP/FI 的前身 (1995)、ASP (1996) 已探索在页面里嵌入服务器端脚本
- Servlet: 在 Java 中输出 HTML; JSP 1.0 (1999): 在 HTML 中嵌入 Java
 - JSP 由容器翻译成 Servlet: 改变的是编写方式
- 页面结构清楚了，业务逻辑也很容易“顺手”写进页面



画面 01:08:00

receiving)

- 避免循环依赖 (如路由不能依赖 DNS)
- 对象自描述 (带类型和大小)
- 没有多份运行代码，就不标准化

4. 命名与地址 (§4)

- 别硬编码地址，用户应用用名字：单一命名结构；公开名字用大小写无关 ASCII；地址在传输起止必须一致。

5. 外部、安全议题 (§5, §6)

- 偏好无专利技术：设计国际化、支持本地化
- 机密性与认证是终端用户自己的责任，别依赖运营商
- 密码算法要可替换、显式标注、留一个强制算法保证互通

一点观察

这份 RFC 的写法本身很值得注意：用“generally felt”“snapshot”“would be foolish to imagine”大量自我限定，把原则写成可演化的启发式而非规范。它甚至主动声明“工程反馈比任何架构原则更重要”——这是罕见的、有自我修正能力的规范文本。

Recap: 用 curl 抓取 RFC 原文后通读，梳理出 RFC 1958 的骨架——通用性为先，端到端/fate-sharing，简单与模块化，避免选项与循环依赖，“有运行代码才标准化”，以及“唯一不变的是变化”这一元原则；并注意到它刻意以启发式而非教条口吻书写。

~/Projects/GenSE (main)
t3.9k 4969 R9.9k CH51.4% \$0.002 0.8%/1.0M (auto) deepseek-flan

```
PROMPT = A + B + ""
...
---
```

```
[No Name] [+]
```

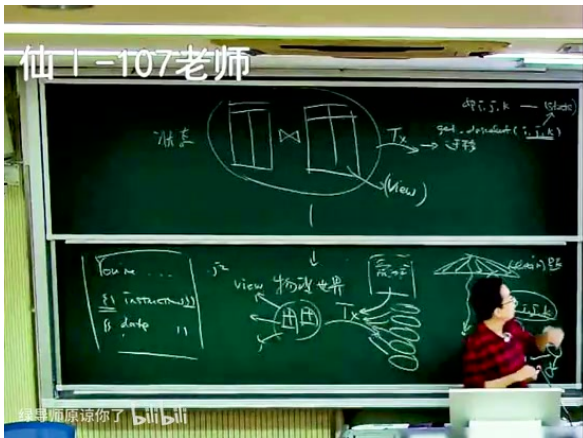
```
-- INSERT --
```

01:08:09

一个更好的办法是用一个模板语言对吧？比如说你把prompt说说写一个you R什么，然后比如说你的instructions，你用一个又是一个语言，这是一个可扩展的语言。然后这个语言的它有一个语义是你可以在渲染这个模板的时候给它传一个叫instructions的变量，然后它会把这个字符串就直接贴在这，这样你的prompt就变得好维护了。如果你有一个Python表达式，你就很难维护掉。比如说你有一个用Python环境一个two的模板，你可以把很多东西，比如说今天的日期对吧？你可能需要今天的日期，你就可以把日期传入传入传传这里。

01:08:56

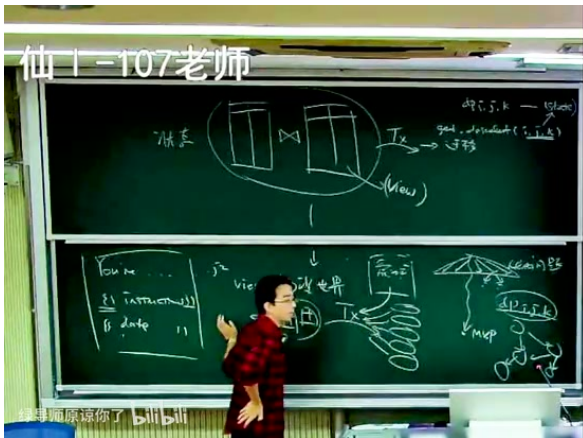
然后这个feature最早大概你看在PP也的时候就已经探索怎么样在页面里面嵌入服务端的脚本了。然后在这个Java时代对吧？Java时代它有对它有几种不同的写法，早期的时候是这样写，很像，对吧？一个一个它可以把Java代码和HTML模板嵌在一起，嵌在一起以后甚至你看这只能起循环。这个像比如说你今天看君家兔里面也有这样的功能，有了这样的功能以后，我们开发就是开发这种以数据库为中心的应用就变得前所未有的容易了，对吧？



画面 01:09:00

01:09:39

我只需要我的每一个数据库应用，尤其是一个页面，我只需要一个database transaction，把所有的我需要的数据都捞出来。比如说存放到一些list里，存放到一些map里。然后他把map就直接用这种方式投射成一个HTML的页面。而里面还有一个很好的设计，它把这个dom的数和样式给分开了，对吧？



画面 01:10:00

Java Web：直接在 HTML 里写代码

Java Web 浪潮 (1990s 后期)

- Java Virtual Machine: Write once, configure everywhere 😊
- 自动内存管理，程序员救星

页面模板：让 HTML 留在 HTML 里

- PHP/FI 的前身 (1995)、ASP (1996) 已探索在页面里嵌入服务器端脚本
- Servlet: 在 Java 中输出 HTML; JSP 1.0 (1999): 在 HTML 中嵌入 Java
 - JSP 由容器翻译成 Servlet: 改变的是编写方式
- 页面结构清楚了，业务逻辑也很容易“顺手”写进页面

JSP：从 Java 片段到表达式与标签

同一张页面，两代写法

- `<% ... %>` / `<% ... %>`: 执行 Java / 输出表达式
- JSTL 1.0 (2002) 提供标准标签与 EL; JSP 2.0 (2003) 将 EL 纳入页面语言

```
<% page contentType="text/html; charset=UTF-8" %>
<% taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html><body>
  <% String msg = "Hello, JSP!"; %> <!-- Scriptlet -->
  <h1><% msg %></h1>
  <ul>
    <c:forEach var="user" items="${users}"> <!-- JSTL + EL -->
      <li><% ${user.name} %></li>
    </c:forEach>
  </ul>
</body></html>
```

- 换成标签，不会自动得到 MVC: 业务规则仍需要独立的归属。

01:10:07

CS和HTMHTML不管样式，但是我可以通过给不同的element不同的样式，我可以排出几乎任何形式的界面，对吧？这样信息系统就可以做了。我可以在这放我再可以在这放button，我再可以在这放任何代码生成的动态的东西。这样你就可以看到为什么像图书管理系统在那个时候就集中式的爆发了。然后在这里面就有了其实是这个想法，就在每个页面都是一个so transaction导出了一个非常重要的模式，叫做MVC model controllers。我们也读一下。

仙1-107老师

receiving)

- 避免循环依赖 (如路由不能依赖 DNS)
- 对象自描述 (带类型和大小)
- 没有多份运行代码, 就不标准化

4. 命名与地址 (\$4)

- 别硬编码地址, 用户应用用名字: 单一命名结构: 公开名字用大小写无关 ASCII: 地址在传输起止必须一致.

5. 外部、安全议题 (\$5, \$6)

- 偏好无专利技术: 设计国际化、支持本地化
- 机密性与认证是终端用户自己的责任, 别依赖运营商
- 密码算法要可替换、显式标注、留一个强制算法保证互通

一点观察

这份 RFC 的写法本身很值得注意: 用“generally felt”“snapshot”“would be foolish to imagine”大量自我限定, 把原则写成可演化的启发式而非规范. 它甚至主动声明“工程反馈比任何架构原则更重要”这是少见的、有自我修正能力的规范文本.

Recap: 用 curl 抓取 RFC 原文后通读, 梳理出 RFC 1958 的骨架——连通性为先, 端到端/fate-sharing, 简单与模块化、避免选项与循环依赖, “有运行代码才标准化”, 以及“唯一不变的是变化”这一元原则; 并注意到它刻意以启发式而非教条口吻书写.

~/Projects/GenSE (main)
3.9k 4969 89.9k CH51.4% \$0.002 0.8%/1.0M (auto) deepseek-fla

```
Pi can explain its own features and look up its docs. Ask it how to use or extend Pi.
```

```
[Context]
~/pi/agent/AGENTS.md, AGENTS.md

[Skills]
compile, lecture-notes, make-artifact, make-slides,
tag-audio, transcribe

[Extensions]
@narunitw/pi-chrome-devtools:dist

Read file://home/jyy/Projects/GenSE/p7/lect/1979-12-MVC.pdf

Thinking...

$ ls -la /home/jyy/Projects/GenSE/p7/lect/ 2>/dev/null; echo
"---"; ls /home/jyy/Projects/GenSE/p7

Working

~/Projects/GenSE (main)
0.3%/1.0M (auto) deepseek-flash • high
```

画面 01:11:00

01:11:04

对，他就是一个非常小的。

01:11:13

一个两页纸的这个页纸，1972年1979年，尤其models represent knowledge对吧？就是一个single point of当前对当前世界的knowledge。谁选的什么课，有哪些订单，然后vi是一个visual representation of its model。所以所谓的will就是我要看到的will是一个function of state，然后这个function是个pure function。所以你不希望这个we有能够有任何改变这个的能力。如果你希望你如果你希望有一个操作可以改变这个世界，你就需要一个controller。A controller is the link between a, the system, it provides user with input by a drain for relevant wheels to present ly placon screen output by priuser with manuals are the meanings of given commands and data.

仙1-107老师

The nodes of a model should all be on the same problem level, it is confusing and considered bad form to mix problem-oriented nodes (e.g. calendar appointments) with implementation details (e.g. paragraphs).

VIEWS

A view is a (visual) representation of its model. It would ordinarily highlight certain attributes of the model and suppress others. It is thus acting as a presentation filter.

A view is attached to its model (or model part) and gets the data necessary for the presentation from the model by asking questions. It may also update the model by sending appropriate messages. All these questions and messages have to be in the terminology of the model, the view will therefore have to know the semantics of the attributes of the model it represents. (It may, for example, ask for the model's identifier and expect an instance of Text, it may not assume that the model is of class Text.)

CONTROLLERS

A controller is the link between a user and the system. It provides the user with input by arranging for relevant views to present themselves in appropriate places on the screen. It provides means for user output by presenting the user with menus or other means of giving commands and data. The controller receives such user output, translates it into the appropriate messages and pass these messages on to one or more of the views.

A controller should never supplement the views, it should for example never connect the views of nodes by drawing arrows between them.

Conversely, a view should never know about user input, such as mouse operations and keystrokes. It should always be possible to write a method in a controller that sends messages to views which exactly reproduce any sequence of user commands.

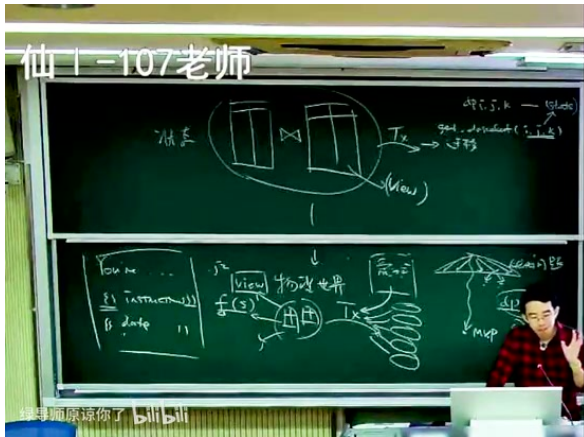
画面 01:12:00

01:12:10

所以他就把这个就是把需求的能修改的点翻译成transaction，然后把你看到东西变成状态的一个投射，非常干净的一个架构。然后像也很自然，它会成为任何信息系统的一个标准的架构。比如说如果你要实现一个教务系统的话，model就是这个课程选课记录各种。然后view就是你看到的那个页面，然后controller就是你各种按钮点下来，或者那输入框点下来，它它相应的会得进入一个数据库的事物。

01:12:46

好，这是大家熟悉的。那这有什么问题呢？这有什么问题呢？问题是你如果如果是在HTML单页面的模式，就只有一个HTML页面的时候。然后每次你点一个按钮以后，这个页面都会重新刷新，重新更新的时候，这个其实是成立的。但是在今天我们的界面上有大量的元素，他们的状态是不在这个model里面的。

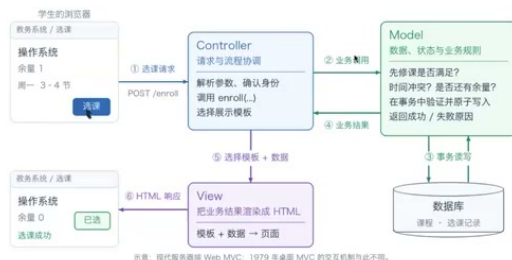


画面 01:13:00

MVC: 一次选课请求

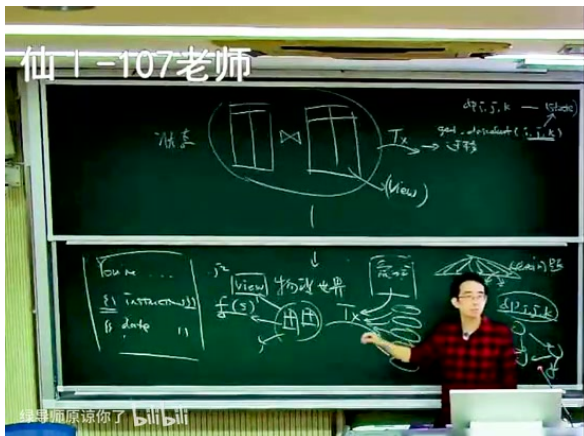
Web MVC 中的业务流程与界面

- 请求处理: Controller 组织请求, Model 执行业务规则, View 呈现结果



01:13:22

比如说我们可以展开随时可以展开一个菜单，对吧？展开菜单或者我有一个搜索框，你们都都用过这个功能吧。呃，我现在有一个视图，有一个view然后这个view比如说是学生的列表，然后这个学生的列表一定是一个搜索框。比如说我要找哪一个名字的同学，找一个对找或者找一个学号。然后教务系统讨厌的地方就是搜索被做成了一个某种一个一个一个database transaction。我需要在搜索框里面先输入，然后输入完了以后点搜索，然后他把这个搜索结果反馈给我。然后有的时候种他已经把这个列表给我了以后，我只需要一个过滤器。然后这个过滤器不涉及任何的model的状态的改变，也不会去再去查这个model。



画面 01:14:00

把 ViewState 也当成 Model

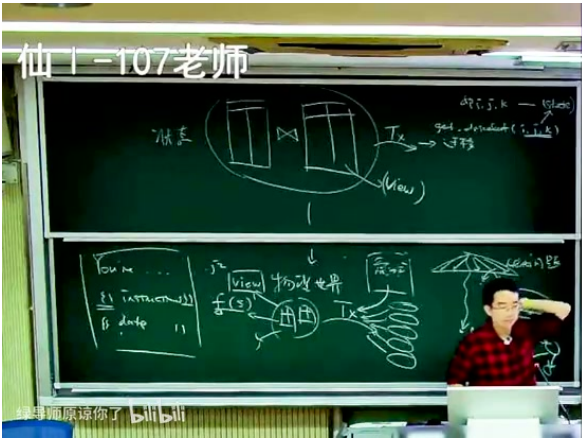
同一份业务数据，不同的界面

- View = render(Model, ViewState)
- 但有一个割裂的地方: ViewState 通常不会进入业务数据库
 - 展开的菜单、hover 的 Element、.....
- 课程数据没变，只改筛选词 query，显示的列表就变了

为什么不把 ViewState 也当成一个 Model 呢？

- Presentation Model (Fowler, 2004): 将界面状态与行为从具体控件中提取出来
- 是否持久化，与是否值得建模，是两件事。

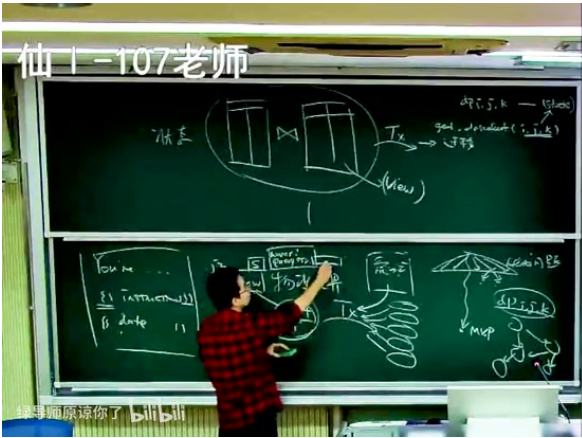
Model已经已经视图已经在视图里了，但我仍然需要对视图的状态做改变。有些我临时的修改我的的那些element，对吧？我的鼠标移过一个元素的，它应该亮起来这些然后这些它其实也需要一个model，但这个model不应该存在数据库里。所以为什么不把real state也当成是一个model呢？然后你就发明了这个MVVM对吧？就现在比较时髦的，就最近比较时髦的一个一个一个一个概念。我看我有没有时间，ok我还有一点时间，我们有两个和MVVM有关系的设计。



画面 01:15:00

01:15:01

其实我觉得我自己我我我不引起胜战，我自己更偏向于这个react。因为react有一个就vi对开发者来说是比较直观的。如果你写过一点前端，你立即就可以上手这个view，然后react来说它更纯粹一点。他希望的是保留这个模型，保留我们我们的也就是说我们看到的東西，网页上面看到东西是整个数据库的一个投射。然后这个是vio上面还可以有一个状态，比如说我是不是ho了，有没有某一个东西被ho了，然后我的query string qustr是什么？它可以有一些自己的页面上面的状态，然后这些状态随时可以改对吧。我有一个我这个qustring是映射到一个文本框的，然后这个文本框和这个的状态是绑定的，当然也可以绑定这个状态。然后这个hover也是的，它每一个每个element都可能都有一个是否hover的however边的状态。



画面 01:16:00

01:16:10

然后他说说好，那我能不能就写一个函数说我的dortree，我把这数据库擦掉，我的domtree就等于function of这个你能不能就直接定义这个函数，我定一个pure function，一个没有任何副作用的函数。我输入这个我输入当前的所有的界面上面的状态，然后我就可以把这个盗墓车给算出来。好，这肯定可以对吧？

React: 用状态快照描述界面

每次 Render 都描述“此刻应该显示什么”

- `const [query, setQuery] = useState('')`: 状态与更新入口
- 组件根据本轮 query 筛选课程、返回 JSX: 派生列表不必另存为 State
- `setQuery` 请求下一次渲染，本轮拿到的状态仍是原来的快照

同一个输入框，显式连接事件与更新

- 状态 → 控件: `value=query`
- 输入 → 更新: `onChange={e => setQuery(e.target.value)}`
- 受控输入框: 事件 → 更新状态 → Render → 界面

React: 用状态快照描述界面

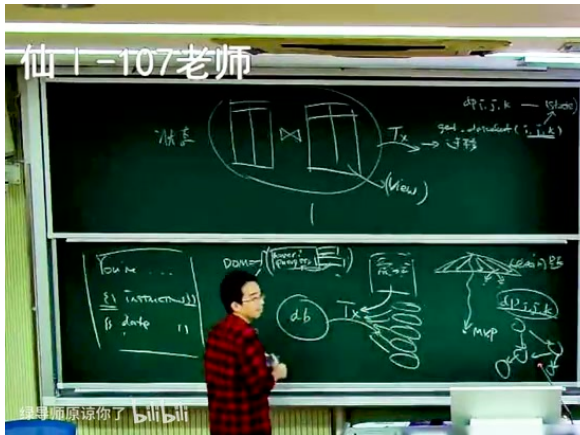
每次 Render 都描述“此刻应该显示什么”

- `const [query, setQuery] = useState('')`: 状态与更新入口
- 组件根据本轮 query 筛选课程、返回 JSX: 派生列表不必另存为 State
- `setQuery` 请求下一次渲染，本轮拿到的状态仍是原来的快照

同一个输入框，显式连接事件与更新

- 状态 → 控件: `value=query`
- 输入 → 更新: `onChange={e => setQuery(e.target.value)}`
- 受控输入框: 事件 → 更新状态 → Render → 界面

可以是可以，但这里有一个非常严重的问题是性能。你的网页上面可能能有几万个元素，然后你当你的这个question改变的时候，开query str当，你当你这里你进入了一个字符的时候，那整个状态全部都要重置。所以他在这个系统里面做了大量的这种virdom的工作，来把这些复杂性给藏起来。你只要按照我的模型来声明什么时候，状态是什么，状态是怎么变的，剩下你就全部都不要管他帮你更新。然后vi有更像是他帮你组建了一个依赖关系图，然后你你你写的时候，当你写的时候你写这样的input等于什么的时候，它就会建立一个依赖关系，然后帮你保持同步。其实两个差不多，而且随着随着好像vivo新版本的发布，他们好像变得更像了。但是我们用的比较少，所以我没有没有这个发言权。



画面 01:17:00

React: 用状态快照描述界面

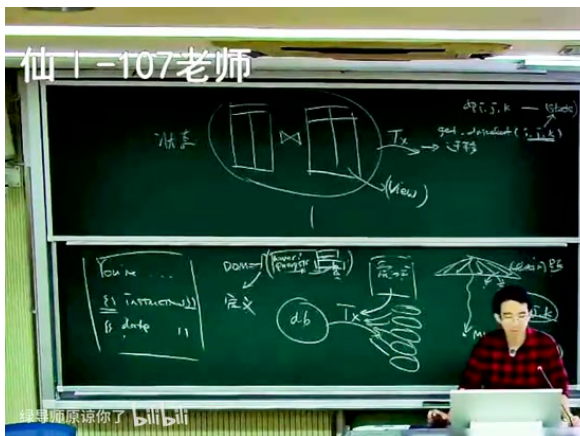
每次 Render 都描述“此刻应该显示什么”

- `const [query, setQuery] = useState('')`: 状态与更新入口
- 组件根据本轮 query 筛选课程、返回 JSX: 派生列表不必另存为 State
- `setQuery` 请求下一次渲染，本轮拿到的状态仍是原来的快照

同一个输入框，显式连接事件与更新

- 状态 → 控件: `value=query`
- 输入 → 更新: `onChange={e => setQuery(e.target.value)}`
- 受控输入框: 事件 → 更新状态 → Render → 界面

9/10



画面 01:18:00

Vue 与 React: 同一个课程筛选框

都由状态决定界面，区别在如何表达更新

- Vue 用绑定语法连接输入与状态; React 在事件中显式调用状态更新函数



⚡ v-model 的双向连接针对绑定: Vue 的组件 Props 与 React 的组件数据流 都是从父到子。

10/10

但你看到在这里就这两个框架，它设计的思路都需要你隔离复杂性。你希望的是你可以把有些变的东西和不变的东西要分离开来分离开来，然后把逻辑和数据的源头要分离开来。这样你可以分开管理，你因为我们是small head，我们头很小。那我们如果关注状态的时候，最好就不要关注这个状态是怎么变成dom m的。然后在我关注到，等我状态完全确定以后，我再看怎么把这个状态翻译成我的dom m对吧？

这样都是由这两个都是由状态决定，界面都是一个pure function，但是更新的模型稍微有一点不一样。好，这是你们现在可能比如说做一些前端开发会比较熟悉的。但是MVCMVVM他们没有解决真正的essential complexity，尤其是面向长流程真正的长流程的任务。

仙 I-107老师

如果是面向客户需求的软件呢？

- 操作系统、CRUD 数据库、Internet 连接了服务器和世界
- 最直接的思路：每个页面就是一个 SQL Transaction
- 就可以支持在线商城、社交网络、教务系统...

Java Web 浪潮 (1990s 后期)

- Java Virtual Machine: Write once, configure everywhere
- 自动内存管理，程序员的救星

一个跨时代的架构设计：****直接在 HTML 里写代码****

[[补充一些这个的历史]]

```

<% page contentType="text/html; charset=UTF-8" %>
<html><body>
  <% String msg = "Hello, JSP!"; %> <!-- 1.0 -->
  <h1><%= msg %></h1>
  <c:forEach var="user" items="${users}"> <!-- 2.0 -->
  <li>${user.name}</li>
  </c:forEach>
</body></html>

```

[Models-Views-Controllers (1979)](1979-12-MVC.pdf): Separation of concerns

p7/lect/index.md 128,0-1 61%

画面 01:19:00

01:19:21

比如说你。假设你要实现一个淘宝，当然你你你不会去实现这么复杂的东西。淘宝的麻烦在哪里呢？你说下单好，下单没问题。然后下单以后他的流程之后会涉及到，比如说下单以后你库存要锁定，锁定完了以后你要计算优惠，然后计算优惠以后你要支付，然后注意这个支付是需要用户确认的，然后支付完了以后你会风控发货、退款、售后对账。所以这个业务流程你用个状态，就刚才我们做了一个其实做了一个不切实际的假设，就是我们用当前状态来表示这个世界。那当然ok没问题，可以用二维表来表示。

仙 I-107老师

应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

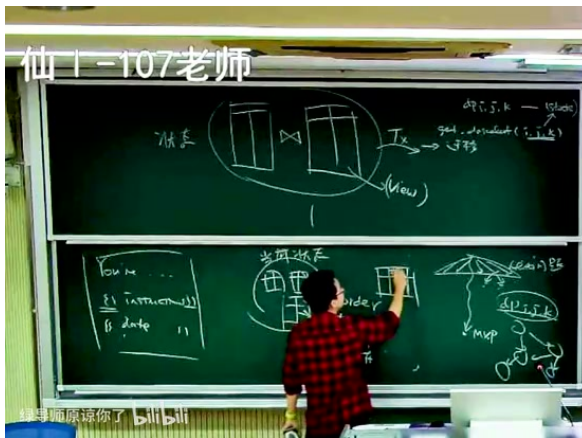
把复杂性放进 Model，不解决 Essential Complexity

- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法

画面 01:20:00

01:20:12

那我在当前状态下，我怎么表示我这个订单到底进行到哪一步了呢？你们最直接的方法就是在这个数据库里面？放一张表叫order。然后order里面这个order这个表里面有一列叫做status，然后series里面有一个有一，再比如有一个一就代表是在这个下单，对吧？如果是一是下单，二是库存，三是什么，或者你可以把它展开来对吧？Order你也可以order展开来，比如说是不是下单，库存支有没有支付对吧？然后比如说它有一个go go go go go好，然后这个时候这个状态机就会变得非常复杂了。而且你你可能遇到的问题是某一个节点上，比如说触发了风控以后，你需要设计它返回的逻辑，然这是一个essential complexity对吧？这是这个业务系统里面的逻辑。



画面 01:21:00

01:21:27

然后随着这个业务系统越长越大，比如不管是这样的管理订单的系统，还是比如说教务系统里面的一个学生，你会发生发现什么呢？你会发现有的学生处于既可以毕业又不能毕业的状态，因为他hack的时候就有的时候你总是在某一个流程上面，比如程序有bug，你的程序有一个bug，你的程序也可能是有一个特批，导致某一个地方被override了。本来这个地方应该封应该触发的，但是其实没有问题的。然后这个风控它就由另外一个人override了。然后在这个系统里面，因为有错综复杂的逻辑，最后就会变成处于一个风控失败的状态。

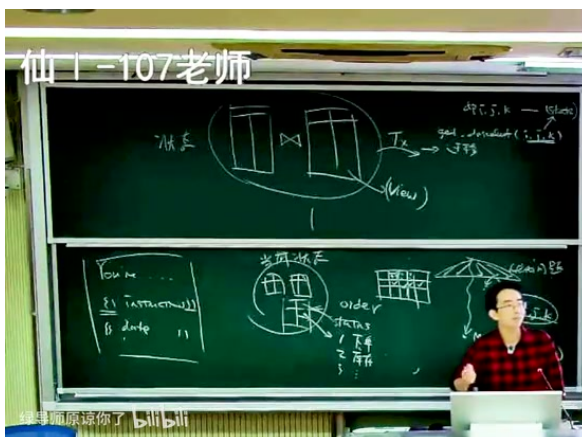
应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

把复杂性放进 Model，不解决 Essential Complexity

- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法



画面 01:22:00

01:22:11

但是它进入了下一个环节，那这个数据到底是什么呢？你怎么解读呢？这种不一致的数据，它甚至有的时候你的数据都是silently corrupt的。我相信我们学校的教务系统里面有无穷多这样。因为经常手操工程师，手操教务系统里面有很多这种就根本就不对不一致的数据在里面。

01:22:33

反正好像最后没有影响这个同学毕业，然后他也就只能只能不管了。这就因为我们的业务流程，它可能是一整条很长的流程。他既要编码过去，又要编码现在，又要编码未来，对吧？所以本质上这是现在的状态的一个快照，但这个快照是由过去推演过来的，他怎么过来的？这个丢了以后你这个一不一致，其实就会就比较复杂。然后我先举一个更简单一点的例子，你们有没有同学记过账？比如说支付宝会给你一个记账，但如果如果你如果你想要真正严肃的记账，它会会计实际上是有一个要求的，叫做复式记账法。那我们也可以。

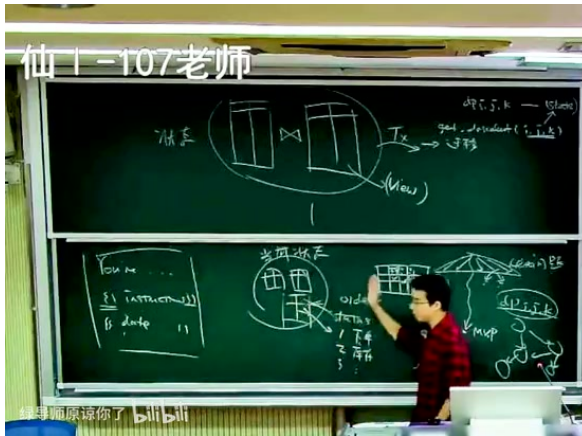
应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

把复杂性放进 Model，不解决 Essential Complexity

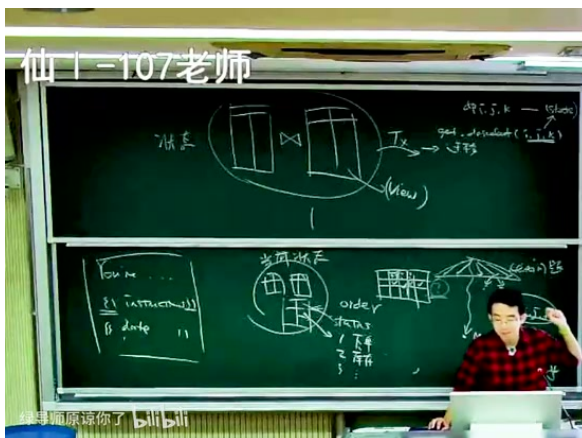
- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法



画面 01:23:00

01:23:32

它是为了防错，它就是为了防止这种数据库不一致。这种不一致就当你发生不一致的时候，他能够检查出来他要求每一笔交易都同时计入两个或者以上的账户，且金额相等，方向相反，对吧？你看他说只记一条，简单，但是无法反映业务的完整结构，也不具备自平衡的校验能力。你看它可以试算平衡它跟人类发明这种东西，就是因为一个当前状态，它很有可能是corrupted是不对的。那他要把这个为什么不对，一定程度上要给呃要要要给要要找出。



画面 01:24:00

01:24:25

对，然后他有一个，你们不需要知道，你们不需要知道这是什么，我也不知道它是什么，但是我有这个记账的需求。然后我有这个记账需求的时候，我当然可以就用这个信息系统的方式来，在做一张一个叫资产负债表的东西，对吧？比如说我可以介绍一下。比如说我现在就有一个需求，我就是记一个资产负债表。然后这个资产负债表大概就是长这样的，有什么现金存货、固定资产、应付账款、短期借款、所有者权益等等。然后这个是相当于如果你要以符合这个会计法规的方式记账，就必须得到一张这样的表。但是我对会计没有任何的支持，那我我要怎么样设计一个软件的架构能够帮我，对吧？在在不知道没有任何知识的情况下，设计一个好的信息系统？

应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

把复杂性放进 Model，不解决 Essential Complexity

- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法

Recap: 通过并这段翻译了 Reenskaug 1979 年 MVC 原始备忘录。核心记忆点是三条职责铁律—模型层表达知识、视图只当展示过滤器且不懂输入、控制器绝不对视图做补充；以及 editor 作为可插拔中间层的思想。

open it with evince

```
$ cd /home/jyy/Projects/GenSE/p7/lect/ 56 (evince
1979-12-MVC.pdf >/dev/null 2>&1 &); sleep 1; pgrep -a
evince (timeout 30s)
```

14196 evince 1979-12-MVC.pdf
14204 /usr/libexec/evince

Took 1.0s

已用 evince 打开 1979-12-MVC.pdf (PID 14196)。

Recap: 用后台方式启动 evince 打开 PDF，并用 pgrep 确认进程存在，避免阻塞终端。

~/Projects/GenSE (main)
11.4k 41.3k R20k CH97.5% \$0.002 0.5%/1.0M (auto) deepseek-fla

短期借款 +1000 (负债增加)

等式两边同时增加，仍然平衡。

为什么重要

1. 自校验：所有账户借贷方发生额必然相等；若对不上，说明有错账，能立刻发现。
2. 可追溯：钱的来源和去向都被记录，任何资金流动都能双向追查。
3. 试算平衡：把所有账户汇总，借贷合计相等是账目正确性的必要条件。

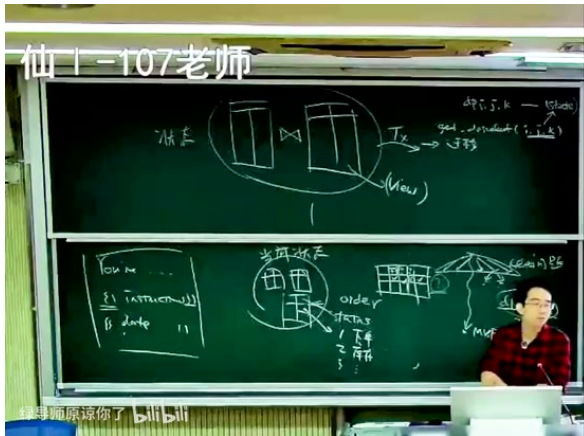
和单式记账对比

单式记账只记「现金收付」一条线，比如「花了 1000 元」，但不记录资产形态变了、负债增加了等等。它简单，但无法反映业务的完整结构，也不具备自平衡的校验能力。

一句话：复式记账把「每笔钱从哪来、到哪去」同时记清楚，用数学恒等式保证了账目天然可校验。

Recap: 复式记账的关键是「借贷相等」这一不变量——它把一个会计问题变成了一个可自动校验的守恒律（类似程序里的不变式 / invariant）。任何一笔业务都在保持等式平衡的前提下重分配数值，因此账目是否正确可以纯粹靠算术自检，而不必透支人工核对。这正是「约束即正确性」思想在现实制度里的经典应用。

仙一-107老师



画面 01:25:00

Recap: 通读并逐段翻译了 Reenskaug 1979 年 MVC 原始备忘录。核心记忆点是三条职责铁律——模型周而表达知识、视图只当展示过速器且不能输入、控制器绝不对视图做补充；以及 editor 作为可插拔中间层的思想。

open it with evince

```
$ cd /home/jyy/Projects/GenSE/p7/lect/ 86 (evinced 1979-12-MVC.pdf >/dev/null 2>&1 &); sleep 1; pgrep -a evince (timeout 30s)
```

14196 evince 1979-12-MVC.pdf
14204 /usr/libexec/evinced

Took 1.0s

已用 evince 打开 1979-12-MVC.pdf (PID 14196)。

Recap: 用后台方式启动 evince 打开 PDF。并用 pgrep 确认进程存在，避免阻塞终端。

~/Projects/GenSE (main)
t1.4k 41.3k R20k CH97.3% \$0.002 0.5%/1.0M (auto) deepseek-fla

所有者权益 = 资产 - 负债

...

包含：实收资本（股东投入）、资本公积、盈余公积、未分配利润（历年赚了没分的钱）。

一张简化表

资产	金额	负债与权益	金额
现金	50	应付账款	30
存货	80	短期借款	70
固定资产	170	所有者权益	200
合计	300	合计	300

两边永远相等——这就是复式记账保证的平衡。

一个直觉理解

把「

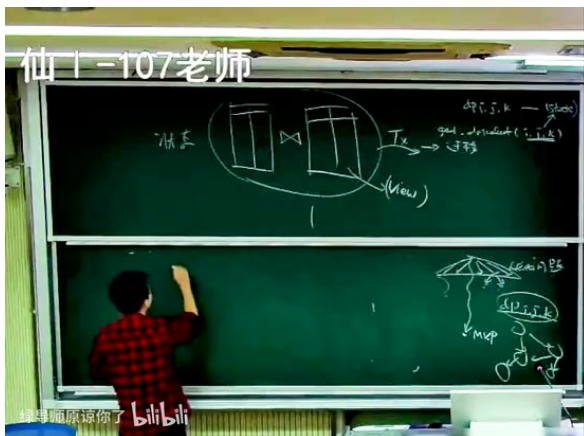
Working

~/Projects/GenSE (main)
t245 4538 R2.9k CH92.3% \$0.001 0.4%/1.0M (auto) deepseek-fla

01:25:31

这时候我就用了一番sourcing我就用了一番，索性我干了这样的一事儿，我不再是记录当前的状态，而是。我记录所有状态的迁移，就是记录这个数结构有很多办法。我可以记录这个数结构当前的状态，我也可以记录这个数结构是怎么怎么来的。所以我其实就是一笔流水账，流水上。比如说今天是收了1000块钱，然后明天又减了1000块钱，然后是因为什么？好，但我不会记账，我没有设计那个数据库系统，没有设置那个数据。而我设计的是我直接让AI I设计了一个这样的东西，我设计了一个小的像编译器一样的东西，然后我有很多的记账的动作，就是我就实际这是这实际中，从最终需要交付的那个资产负债表里面所有可能的条目里面提取出来的。

仙一-107老师



画面 01:26:00

记账架构：Event Sourcing & LLM as a Compiler

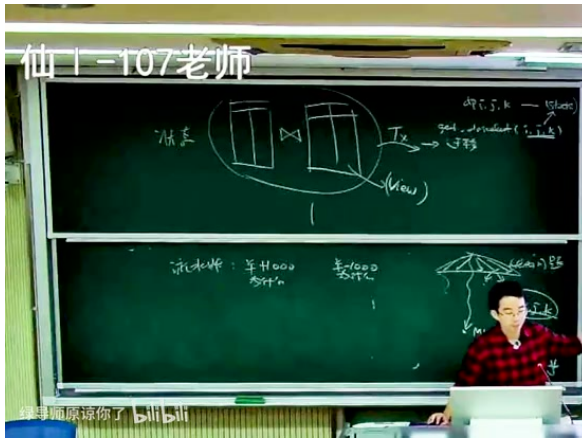
从原始凭证到可执行的分录

- 原始凭证 (事件) → 分录 (报表支出_财务费用(CNY(75)), ...) → State Change
- 借贷平衡是可检查的约束，不代表业务真实：分录仍须核对凭证。试算平衡的局限

```
@Curry
def pay_payroll(state, category, amount, *, kind):
    """按实际付款金额结算薪酬或个人劳务：代扣款先单独记账，再支付净额。"""
    flow = PAYROLL_FLOWS[category]
    key = payable(flow, kind == "个人劳务")
    state.require(key, amount)
    cash(state, -amount, flow, balances={key: -amount})
```

01:26:45

然后比如说我要我要我要付钱，比如说我要付一个营业成本的钱，那每付出一份我都会在这些表格上面。比如说货币资金会怎么样，所有者权益会怎么样，这个应付账款会怎么样，然后在现金流量表上会发生什么？然后你看到这个表格完全是AI I生成的，我就选择相信AI I对吧？选择相信AI I以及相信最终最终那个校验会找到这里面是不是会有任何的问题。然后接下来我做了一个什么样的架构设计呢？我做了一个什么样价格设计呢？

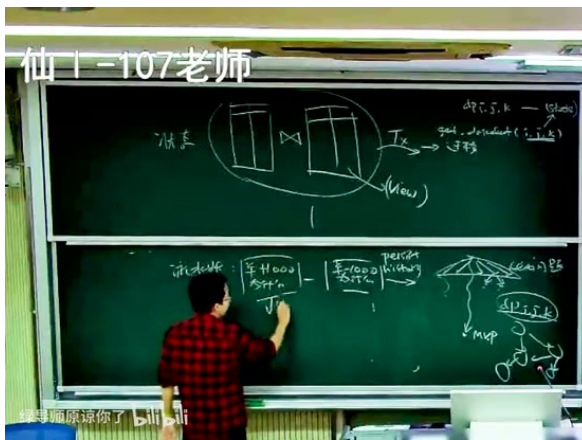


画面 01:27:00

01:27:31

我用LM做编译器，就这这是一个persist history，就是不可篡改的就发生了什么。比如说我今天我要我要允许某一个不能毕业的同学毕业，我也必须要在里面写一个意思说我要允许什么毕业。然后这样的一个流水账的条目会用LRM编译成刚才你们看到的那一个小的操好多小的操作。然后当前的状态是由所有的这个操作reduce得到的，就相当于每一个操作都会说好我的这个资产负债表要怎么改，第二张表要怎么改，现金流量表要怎么改。然后每一个操作最后把它replay一遍就得到了当前的我可以比如说提交出去的这个资产负债表，当然在这这是这个总体的架构，然后这里面还有很多小的架构。比如说我这里用了一个cover呢g写的代码很糟糕，就一开始写的非常长的代码，然后我就用了一个小的trick把它合并起来。我可以给这个payroll加上一个参数，然后就它就可以得到一个payroll，相当于是这个参数state已经确定了，然后剩下的这样的一个函数一个函数编程里面非常常用的小的trick。

净利润	资产负债表		利润表		现金流量表
不重复确认费用,	流动负债合计	+1.00	税收滞纳金		+1.00
	负债合计	+1.00	利润总额		-1.00
	未分配利润	-1.00	净利润		-1.00
计提的利息须结算	所有者权益合计	-1.00			
不重复确认费用,	货币资金	-1.00	不变		购买原材
	流动资产合计	-1.00			劳务支付
	资产合计	-1.00			经营活动
	应付账款	-1.00			净额
计提的利息须结算	流动负债合计	-1.00			现金净增
	负债合计	-1.00			期末现金
	负债和所有者权益总计	-1.00			



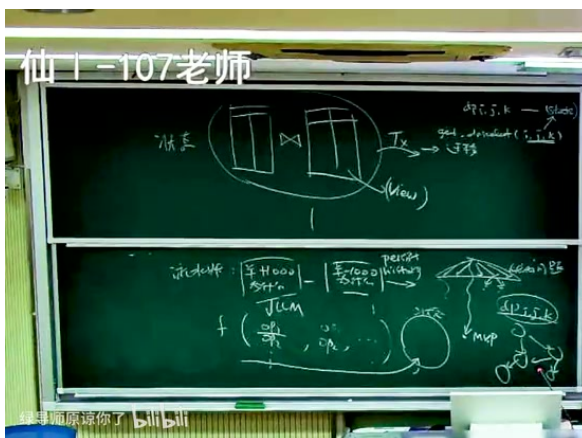
画面 01:28:00

记账架构：Event Sourcing & LLM as a Compiler

从原始凭证到可执行的分录

- 原始凭证 (事件) → 分录 (报表支出_财务费用(CNY(75)), ...) → State Change
- 借贷平衡是可检查的约束，不代表业务真实：分录仍须核对凭证。试算平衡的局限

```
@Curry
def pay_payroll(state, category, amount, *, kind):
    """按实际付款金额结算薪酬或个人劳务：代扣款先单独记账，再支付净额。"""
    flow = PAYROLL_FLOWS[category]
    key = payable(flow, kind == "个人劳务")
    state.require(key, amount)
    cash(state, -amount, flow, balances={key: -amount})
```



画面 01:29:00

记账架构：Event Sourcing & LLM as a Compiler

从原始凭证到可执行的分录

- 原始凭证 (事件) → 分录 (报表支出_财务费用(CNY(75)), ...) → State Change
- 借贷平衡是可检查的约束，不代表业务真实：分录仍须核对凭证。试算平衡的局限

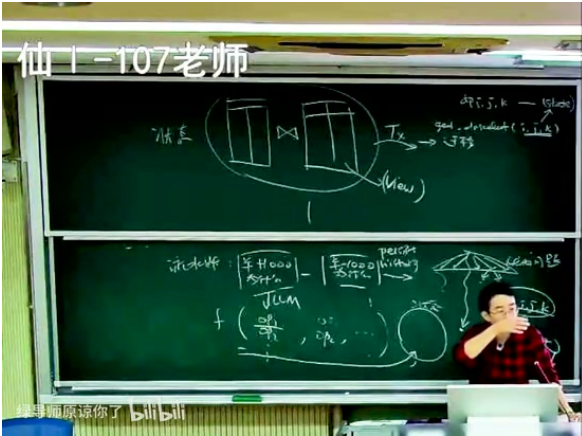
```
@Curry
def pay_payroll(state, category, amount, *, kind):
    """按实际付款金额结算薪酬或个人劳务：代扣款先单独记账，再支付净额。"""
    flow = PAYROLL_FLOWS[category]
    key = payable(flow, kind == "个人劳务")
    state.require(key, amount)
    cash(state, -amount, flow, balances={key: -amount})
```

01:29:09

然后好处就是记账就不再是任何问题了。我可以把比如说这是一个实际上记账发生的条目，比如说我要银行收支，比如说隔一段时间就会银行收支会要整理一下。那它在报表上面体现的所有的side effect都会对吧？都会体现在这里。

01:29:36

然后我把它全部都按时间顺序一遍，就得到了当前的当前的报表，这是一万。然后从这个例子里面，我们看到什么呢？如果我们设计一个信息系统，就我刚才说如果你用这个database去管理信息，那你的database很自然的，你管理的是现在对吧？



画面 01:30:00

记账为什么麻烦？

“当前状态”同时编码了过去、现在、未来

- 过去：合同签订
- 现在：等待打款
- 未来：货物交接、潜在坏账.....

01:30:01

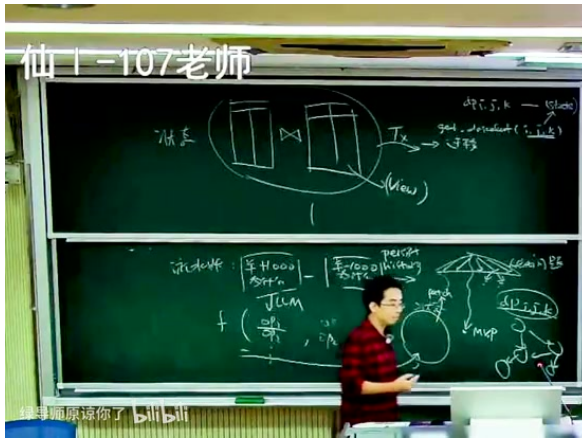
现在学校里面的状态是什么？每一个学生是什么？我们学校里面有哪些课程对吧？每个学期有哪些课程？它代表一个当前的事实，然后这个状态是可变的。状态可变就意味着你一旦你的逻辑有错，逻辑改变你状态就很容易造成不一致。你会从一个对的状态走到一个错的状态。

01:30:29

而刚才我们讲什么样的好价构是好的呢？你能够生长的价构是好的，对吧？能够容纳。比如说突然多了一个多了一个新的需求，或者过去的某一个规则改掉了，然后当过去的规则改掉的时候，你这个时候你就需要为状态生成一个patch对吧？你就需要为状态生成一个补丁，然后这个状态的补丁数据库的迁移就是一个非常危险的操作。

01:30:58

因为你你到底决定？比如说培养方案改了错了，修正了，成绩更正了。成绩更正会带来大量的后果。比如说你的成绩从及格到不及格。打个比方，这个老师假设这位同学你想去要分，然后老师说你不要要了，然后你非要查卷子。然后查了卷子以后，老师发现当时救了你一手，本来你最后那个题空着的，还给了你20分。然后好，那这样好了，他查了卷子只能把这20分扣掉，然后就挂了。



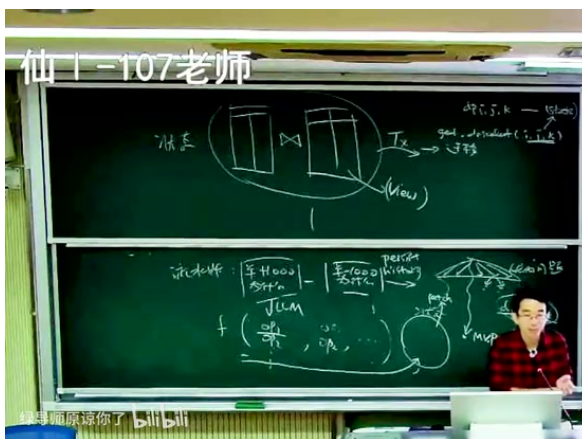
记账为什么麻烦？

“当前状态”同时编码了过去、现在、未来

- 过去：合同签订
- 现在：等待打款
- 未来：货物交接、潜在坏账.....

01:31:35

然后挂了以后，他对这个数据库就是一个一个一个修改，一个可能就是不在原来业务逻辑里面的修改，对数据库是摧毁性的。因为你这门课从挂了以后，你的学分数要减1，然后你的学分数又会引起你能不能毕业，能不能准出，能不能选下一门课。就是你的数据库内部其实已经不一致了。打个比方，你已经选上了它的先修你要选修操作系统，你要先修计算机系统基础，然后你计算机系统基础通过了，所以你选上了操作系统。然后你现在计算机系统基础挂科了，你还能选操作系统吗？它内部它天生就是不一致的，这个以sential complity里有大量的这种依赖关系，我依赖你你你依赖我。



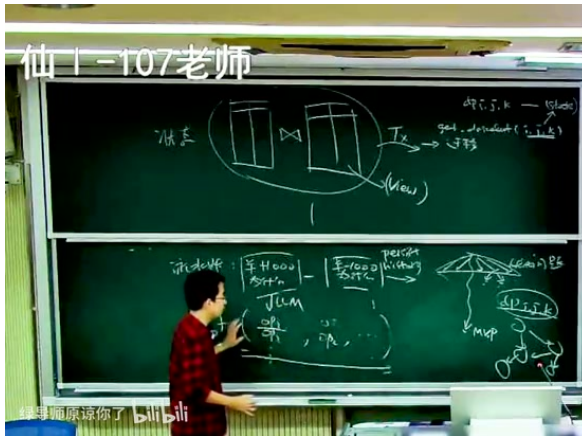
记账为什么麻烦？

“当前状态”同时编码了过去、现在、未来

- 过去：合同签订
- 现在：等待打款
- 未来：货物交接、潜在坏账.....

01:32:27

好，那这个时候如果是一个当前状态的database，好的，你这个课好像这个成绩对吧？成绩和是否挂科这一栏你就从一改成0。但是你对数据库剩下的部分你到底做了什么？你是很难评估的，很容易漏掉，很容易。所以为什么要用event sour性呢？Event sour性是说你不要你做的事情不是维护一个当前状态，而是你要搞清楚所有的原始数据之间的dependency。



画面 01:33:00

记账为什么麻烦？

“当前状态”同时编码了过去、现在、未来

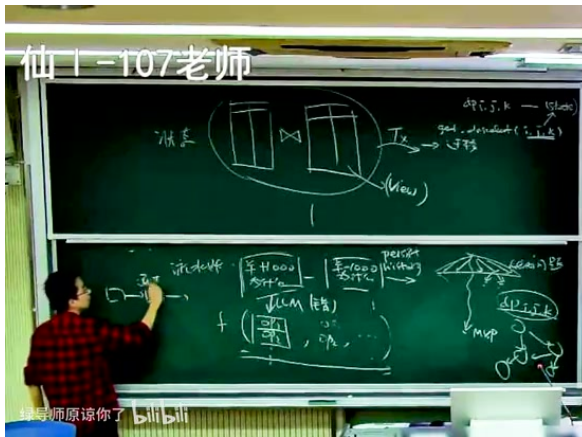
- 过去：合同签订
- 现在：等待打款
- 未来：货物交接、潜在坏账.....

01:33:02

比如说从一个这样的LM编译器，从一个记账条目到一个操作，这也可能会错的对吧？这也可能会错的。但是错了以后，比如说我当期的这个账，我我我会有人工来审核审核。如果发现这个错了以后，我可以把它修改过来，我不会付出任何的代价，对吧？这个翻译错我就直接把它改正就可以了。然后如果我真的发生了一个，比如说像刚才我说的选课选课选课这个选课你影响其他的课程，你在这样的架构里面你就会知道，对吧？你改了这个事实，比如说改了，然后我还换一个对吧？

01:33:53

比如说选课，比如说你你你通过比如这个同学通过了这个IS课，然后通过了S以后，有很多东西事后算出来的。然后当这个东西发生了反转以后。因为你不维护当前的状态，你有所有的计算，你可以把这个计算重新全部做一遍。你哪怕用一个自动的数据流分析，你都知道通过S从通过到不通过，它可能会影响哪些结论，因为你把data follow就make explicit了，而不是以状态的形式藏在那个当前状态里。而且one还有很可以带来很多非常好玩的特点。因为它是这个replay base的，它是replay based，你我当前的状态完全是由过去决定的，所以我可以用它来推演未来的状态。



画面 01:34:00

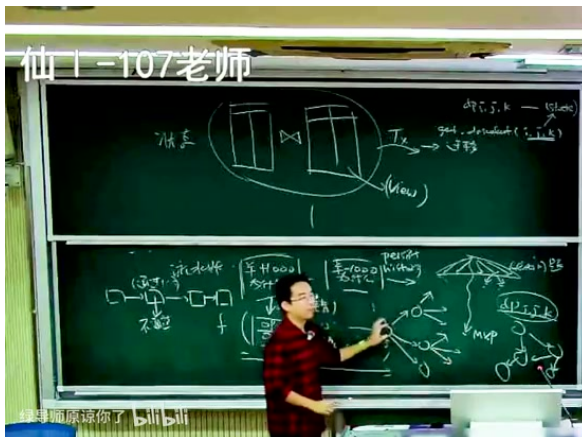
Event Sourcing 和复杂性分解

把事实、视图、预期分开

- 过去是事实，不可更改
- 现在是视图，是事实经 dependency 计算得到的
- 未来是预期，不是事实
- 如果世界没有按预期发展，就追加新的事实来矫正

01:34:49

所以对我来说，我的记账系统最大的好处实际上是我可以假设，比如说打比方这个流程里下单锁库存优惠计算，我现在在支付上，那么我现在可以在这个位置上直接推演出，就因为这个价格天生就支持，我可以推演出如果风控成功状态会到什么样？如果风控失败状态会怎么样。也就是说如果我有账本，我其实我可以在这个event source的基础上去评估我到底会不会破产，我破产的概率有多少，我只要有一个模型，我只要知道比如说我应收账款有多少回不来，我就会破产，我就能评估我的这个？为对未来做这样的评估，这都是一个好的架构所能够带来的所以event sourcing为什么在解决这些问题的时候，会显得稍微要好一些。它没有办法消除这些essential complity，就这些依赖关系，我的先修课程的依赖关系，我的毕业的依赖关系，这些都是可以都是事实已经存在的，但是event把事实视图和预期它分开了。然后如果世界没有按照你的预期发展，你就通过追加新的事实，就你不需要让那个工程师去手改一个数据库，然后让这个数据库进入越来越糟糕的状态，而是你可以追加一个。我现在领导认为这个同学就是可以毕业，其所有的东西都不管了，你就让他毕业。然后因为有这个偶的，所以可以他他可以他可以闭业。

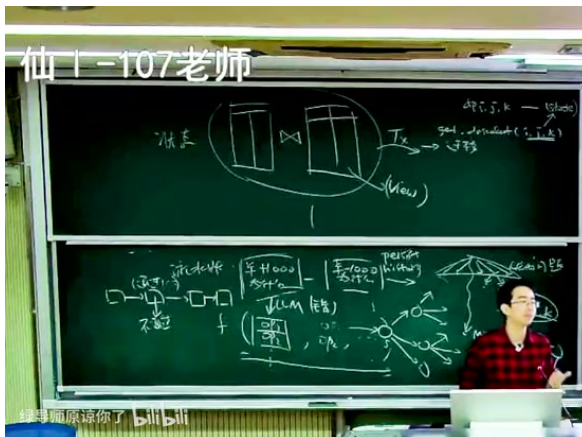


画面 01:35:00

Event Sourcing 和复杂性分解

把事实、视图、预期分开

- 过去是事实，不可更改
- 现在是视图，是事实经 dependency 计算得到的
- 未来是预期，不是事实
- 如果世界没有按预期发展，就追加新的事实来矫正



画面 01:36:00

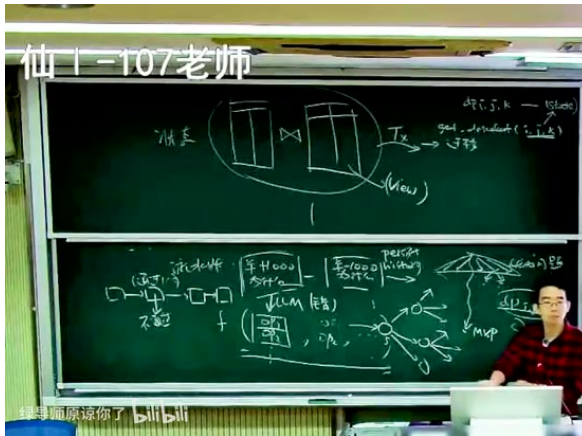
Event Sourcing 和复杂性分解

把事实、视图、预期分开

- 过去是事实，不可更改
- 现在是视图，是事实经 dependency 计算得到的
- 未来是预期，不是事实
- 如果世界没有按预期发展，就追加新的事实来矫正

01:36:38

所以如果你你这个跳出问题，看问题的话，就一句话，现在是过去的结果。所以你有一个可查证可溯源的计算过程，你就显示的建模了这个系统里面各种各样的data dependency。虽然你没有就用一个数据结构去对吧？一个是data dependency graph。我这个数据是你可以在当前数据库里面说，我这个表的这个数据是依赖这个的这个是依赖这个的这个是依赖这个的，你没有做这样的，你还是一个计算逻辑，还是说我我因为有因为我我有这个通过了SS所以我这个学分要加加几？要加上这个SS的学分，我要从这个时候，通过SS课我就要取出在这个时刻SS课的学分。然后我知道这个学分是几以后，我就可以把它加上去。所以所有的这个计算逻辑都是可以溯源的？因为计算机系统基础课的学分可能是变的，你如果有一般sour你就知道在当前这个时刻，计算机系统基础课的学分是几，你就不会出现对吧？确实我们减了学分，然后那你学分还不够不够毕业的毕业的问题。



画面 01:37:00

01:37:56

对所以在这个意义上，我觉得si口或者说关系代数它犯了一个billion dollar mistake。这个来自于Tony霍尔，他调侃自己他发明了空指针，然后他调侃自己是一个billion dollar mistake。因为有no point，就个程序crash，crash和火箭可能也爆炸了这样。

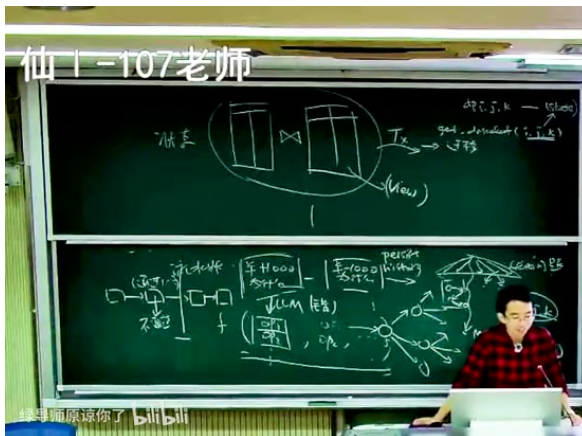
跳出问题看问题：“现在”是“过去”的结果

显式建模 Data Dependency

- 选课记录 / 成绩表 → 当前已修课程集合、学分数
- 已通过课程集合 + 培养方案 → 是否满足课程准出的毕业条件

一个可查证、可溯源的计算过程

- 解决“反悔将导致多处修改，遗漏一处即崩溃”的问题
- 复杂性仍然可能带来不一致，但容易发现、容易修复



画面 01:38:00

01:38:21

然后其实关系代数的一个也我也不能讲设计缺陷。就在那个时候，它当然是一个非常可以扩展的架构，非常好的架构。但他为了管理数据的时候，他没有把任何图灵可计算的计算关系作为1st class citizen管起来。这个是religidatabase，它天生做的不是那么好的。也就是说如果我的database里面有两个，有两个field的，就像我能不能毕业对吧？就比如说学生里面有一个能不能毕业，然后这个毕业零或者一数据库几乎都会这样设计。因为你必须要快速的把它查询出来，对吧？它不能用关系大数据建模一个非常复杂的试图，建模一个非常复杂的计算，那么在这个时候这个零和一就成为了一个数据库里面不一致的潜在的点。

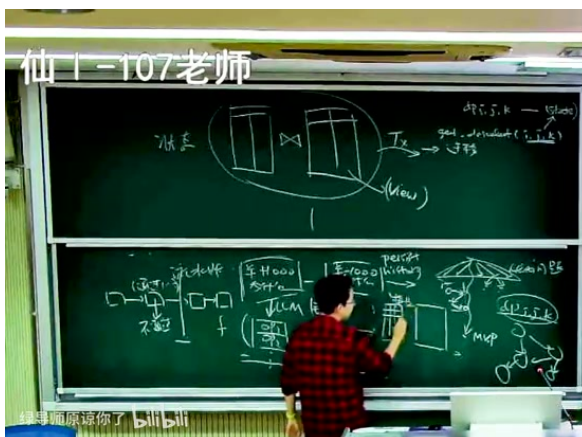
SQL 的 Billion Dollar Mistake 🤔

一个设计缺陷：任意计算关系没有成为 First-Class Citizen

- SQL 没能把跨数据的任意计算关系作为 first-class citizen
- SQL 已有视图 / 物化视图：这里追问的是：跨数据库与应用代码的任意计算依赖，谁来统一维护？

现代框架以不同方式找补

- MVVM、React、Dataflow、Event Sourcing.....
- 把 derived state / dependency 显式化



画面 01:39:00

SQL 的 Billion Dollar Mistake 🤔

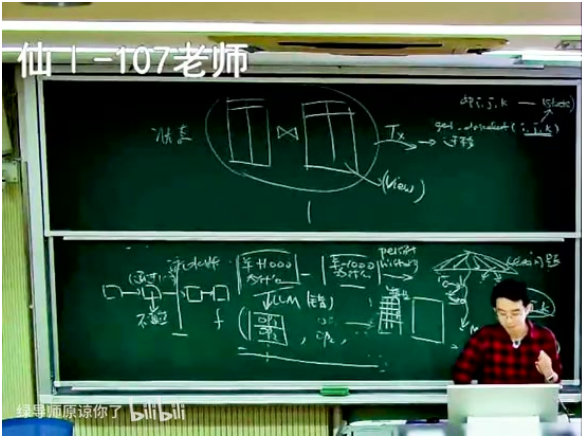
一个设计缺陷：任意计算关系没有成为 First-Class Citizen

- SQL 没能把跨数据的任意计算关系作为 first-class citizen
- SQL 已有视图 / 物化视图：这里追问的是：跨数据库与应用代码的任意计算依赖，谁来统一维护？

现代框架以不同方式找补

- MVVM、React、Dataflow、Event Sourcing.....
- 把 derived state / dependency 显式化

然后现在的框架用各种不同的方式来找补。你会看到这个MVM react data flow event都是试图把这种derive state和dependency显示化。所以我觉得去看一看这个老旧的东西，然后再去理解。比如说新设计为什么我们要做做一般会就会觉得比较有意思。



画面 01:39:40

从状态一致性到计算正确性

保存足以重建其余信息的 Source of Truth

- 其他状态尽可能作为 computation 的结果
- 复杂性没有消失，但从 **state consistency problem** 变成了 **computation correctness problem**

计算可以重跑、测试、审计、依赖分析

- 逻辑错误也更好补救：修正规则，从某个点开始使用新规则计算
- 复现当时的结果，需要当时的规则与外部输入：哪一年的培养方案、哪一天的汇率？[Event Sourcing](#)
- 重放可以重算余额，不能再次打款。隔离外部副作用